Ministry of Higher Education and Scientific Research

University of Manouba

National School of Computer Science

Graduation Project Report

Presented in order to obtain

National Diploma of Engineering in Computer Science

SUBJECT :

Design and development of a Quality Management System

Focus

Elaborated by : Issam Mbarek
Monitored by: Madame Manel Smati

Supervised by: Madame Houda Guermazi

Hosting company: FOCUS

Adresse: Lot numéro 33 Zone Industrielle Chotrana2, Ariana.

Tel: 216 70 106 100 **Fax:** 216 70 106 111 **Email:** focus@focus.com.tn

Academic year :

2013 - 2014

ملخص

يغطي هذا التقرير تصميم وتطوير المشروع الذي سيساعد الشركة على تعزيز إدارة الجودة. فمن المفترض أن تحظى استخدام وعرض مؤشرات الأداء الرئيسية في لوحة أجهزة القياس بسيطة و سهلا على المستخدم كما ينبغي أن إدارة المراجع من الوثائق والتعامل مع عملية التدقيق.

وقد تم تطوير التطبيق باستخدام هندسة ال- J2EE باستخدام الأطر Spring و Hibernate. وتم تحزيز الجانب العميل عن طريق JSF2، HTML5 وPrimeFaces لقد تم هذا المشروع في شركة FOCUS داخل قسم إدارة الجودة وهو مشروع ختم الدراسة.

الكلمات المفاتيح : عملية التدقيق ، مؤشرات الأداء الرئيسية ، أجهزة لوحة القياس J2EE, Spring, Hibernate, PrimeFaces, JSF2 .

Résumé

Ce rapport couvre la présentation du développement et la conception du projet de fin d'études PFE. Ce projet a comme but d'augmenter la performance de la gestion de qualité au sein de la société FOCUS. Cette application doit rendre la gestion des KPI plus efficace et plus simple à travers une Dashboard. Elle doit aussi faire la gestion des références des documents, et le planning des Audits. Cette application a été développée autour l'architecture J2EE, avec l'intégration les Frameworks Spring et Hibernate et en utilisant dans la coté client PrimeFaces, JSF2 et HTML5. Ce projet a lieu au sein de l'entreprise FOCUS dans le département de la gestion de la qualité.

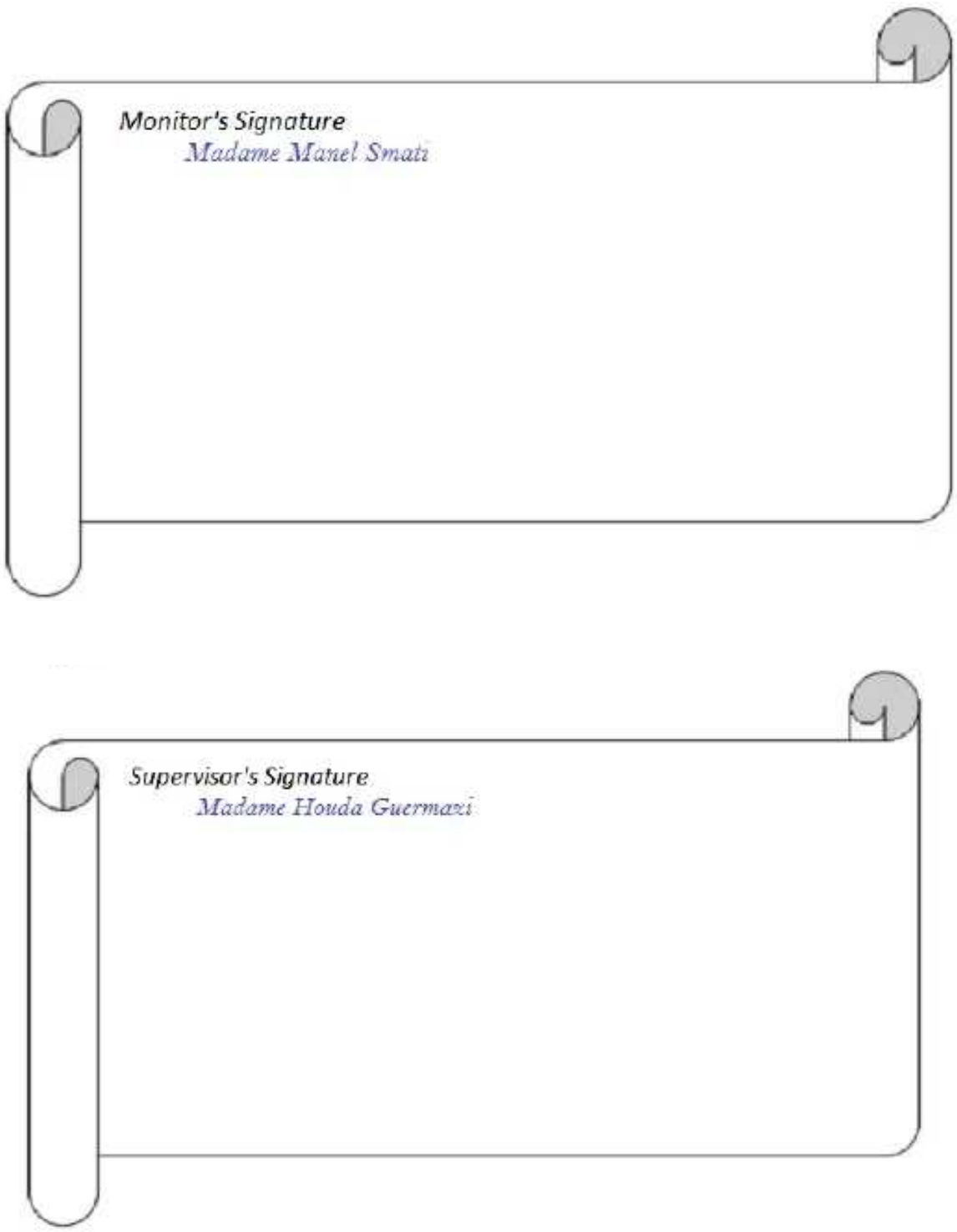
Mots clés : J2EE, Spring, Hibernate, PrimeFaces, JSF2, Dashboard, KPI, Audit.

Abstract

This report covers the design and development of a project that helps the company to enhance its Quality Management. It is ought to make the use and display of the KPIs in a simple dashboard easier for the user. It should manage the references of documents and handle the Audit process. The application was developed using the J2EE architecture using the Spring and Hibernate frameworks. The client side was enhanced by the powerful JSF2, HTML5 and PrimeFaces. This project took place in the Focus within the Quality Management department as an end of study project (PFE).

Keywords: J2EE, Spring, Hibernate, PrimeFaces, JSF2, Dashboard, KPI, Audit.

Signatures and stamps



Dedication

Hoping that it meets their expectations, i sincerely dedicate this modest work to all the people that believed in me, and gave me the chance to be myself.

Namely my dear parents Chelbia Bent Chelbia and Aissa Mbarek, my dear brother Anis Mbarek and all of my friends, the true ones.

Acknowledgment

During my experience of internship in Focus i got acquainted with many memorable people whose help was such a treasure for me. Thus, I will not miss this opportunity to thank them and express my deep gratitude and respect.

I start by Madame Manel Smati, my company's monitor, who with her patience, help and guidance i was able to achieve my work. I want to thank her for helping me in understanding the work process in the management department of the company.

I want to express my gratitude for all the members of the "SAP development" team and the other workers from other teams and departments who, for some of them, made me feel welcomed and accepted in the company, and for some others, who actually made me feel like I was at home.

I want to thank Madame Houda Guermazi, my supervisor, for her help and valuable advices during the phase of writing the project's report. I want to thank her also for her availability, believing in me and my capabilities, and patience in correcting, checking and reviewing my report.

I would like also to thank all of my teachers at ENSI for their continuous help and treasurable training during my study years. And I thank the jury members who gave me the great honor of examining and evaluating this modest contribution and took the time to read my report.

Contents

General introduction **1**

1 Project's frame **3**

1.1 Presentation of the hosting Company 3

1.1.1 Generalities and mission of Focus 3

1.1.2 History of Focus 4

1.1.3 The Quality Management in Focus 5

1.2 Project overview 5

1.2.1 The reasons behind the project 5

1.2.2 The Aim of Our Work 6

1.2.3 The adopted work methodology 6

2 Preliminary study **8**

2.1 Study of the state of the art 8

2.1.1 literature review of quality in outsourcing 8

2.1.1.1 Importance of the quality in the outsourcing 9

2.1.1.2 Computerizing the adopted quality management system 9

2.1.2 The project's Key words 10

2.2 Assessment of the current working method in Focus 12

2.2.1 KPI management and tracking 13

2.2.2 References generation 16

2.2.3 Audit planning 17

2.3 Critics of the current way of work 18

2.4 The proposed solution and expected work 18

3 Requirements analysis and specification **20**
3.1 Requirement Analysis . . . 20
3.1.1 Functional requirements . . . 20
3.1.2 Non-functional requirements . . . 21
3.2 Requirement specification . . . 21
3.2.1 Identification of the actors of the application . . . 21
3.2.2 Use case Diagrams . . . 22
3.2.2.1 General use case (system's use case) . . . 22
3.2.2.2 Extended "Plan the audit" use case . . . 23
3.2.2.3 Extended "Manage the dashboard" use case . . . 24
3.2.2.4 Extended "Manage the KPIs" use case . . . 25
3.2.3 Description of the most relevant nominal scenarios . . . 26
3.2.3.1 Nominal scenario for documents traceability . . . 26
3.2.3.2 Nominal scenario for Audit planning . . . 27
3.2.3.3 Nominal scenario for KPIs tracking . . . 28

4 Design and structure **31**
4.1 Global Design . . . 31
4.1.1 Logical architecture . . . 32
4.1.2 Applicative architecture . . . 35
4.1.3 Technical architecture . . . 36
4.2 Detailed Design . . . 38
4.2.1 Model layer (data's model) . . . 38
4.2.2 Data access object (DAO) layer . . . 40
4.2.3 Service layer . . . 41
4.2.4 Presentation layer . . . 42

5 Achievements **44**
5.1 Developing environment . . . 44
5.1.1 The hardware environment . . . 44
5.1.2 The software environment . . . 45
5.1.2.1 Main Programs . . . 45
5.1.2.2 Other Programs . . . 46

5.2 Technical choices: Development languages and paradigms 46
5.2.1 Development languages 46
5.2.2 Development standards 47
5.2.3 Development frameworks 47
5.2.3.1 Spring Framework 48
5.2.3.2 Hibernante Framework 49
5.2.3.3 PrimeFaces Library 49
5.3 Work achievements 50
5.3.1 Access to the application 50
5.3.2 Administration of the application 51
5.3.3 Managing the KPIs 55
5.3.4 Document traceability 57
5.3.5 Audit planning 57
5.4 Project's planning 58

General conclusion and perspectives **60**

Bibliography **63**

Nethography **64**

A ISO 9000- Quality management **66**
A.1 ISO 9001:2008 66
A.2 Support for implementing ISO 9001:2008 67

B Capability Maturiy Modal Integration: CMMi **68**

List of Figures

1.1 The logo of Focus 3

2.1 Example of dashboard from Oracle Corporation 12
2.2 Information related to the KPIs management 14
2.3 Update of a values for a KPI 15
2.4 sample of a referencing document 16
2.5 Sample of audit planning 17

3.1 System use case 23
3.2 Extended "Plan Audit" use case 24
3.3 Extended "Manage the dashboard" use case 24
3.4 Extended “Manage the KPIs” use case 25
3.5 Nominal scenario for documents traceability 27
3.6 Nominal scenario for Audit planning 28
3.7 Nominal scenario for KPIs tracking 29

4.1 Spring and Hibernate architecture 32
4.2 The application’s logical layers 33
4.3 The application final design 35
4.4 Package diagram for the application 36
4.5 The technical design of the application 37
4.6 Class diagram for the Model layer 39
4.7 Class diagram for the DAO layer 40
4.8 Class diagram for the Service layer 41
4.9 Class diagram for the Presentation layer, data package 42
4.10 Class diagram for the Presentation layer, presentation package 43

5.1 Sum up diagram for the choosen frameworks 48
5.2 Login page 50
5.3 Home page 51
5.4 Admin page 52
5.5 Interface to add new information 52
5.6 Interface to display the existing information 53
5.7 Interface to delete information 54
5.8 Interface to update information 54
5.9 Dashboard settings 55
5.10 Display of the Sales turnover's value for the year 2012 56
5.11 Interface to update the KPIs values 56
5.12 Interface to provide the information for document tracking 57
5.13 Interface to audit planning 58
5.14 Work timeline 59

Acronyms

ACL	Access Control List
API	Application Programming Interface
CMMI	Capability Maturiy Modal Integration
CRUD	Create-Read-Update-Delete
CSS	Cascading Style Sheets
DAO	Data Access Object
IoC	Inversion of Control
IDE	Integrated Development Environement
IT	Information Technology
ITO	Information Technology Outsourcing
ISO	International Standardization Organization
HQL	Hibernate Query Language
HTML	Hypertext Markup Language
J2EE	Java 2 Enterprise Edition
JDBC	Java Database Connectivity
JPA	Java Persistence API
JSF	JavaServer Faces
JSP	JavaServer Pages
KPI	Key Performance Indicator
LDAP	Lightweight Directory Access Protocol
ORM	Object Relational Mapping
PA	Process Area
POJO	Plain Old Java Objects
QM	Quality Management

QMS	Quality Management System
RDBMS	Relational Database Management System
RUP	Rational Unified Process
SME	Small and Medium Enterprises
SPI	Software Process Improvement
SQL	Structured Query Language
SRS	Software Requirements Specification
UML	Unified Modeling Language
UP	Unified Process
XHTML	Extensible HyperText Markup Language
XML	Extensible Markup Language

General introduction

Over the last years, the competition between companies has become harder than ever before. Especially for the IT-outsourcing-based companies. And in order to stand out and have a better share of the market, many companies try to make their internal processes and work more efficient and valuable.

That's where the set of theories and principles of the Quality Management, which will be annotated throughout this report by QM, happen to be a valuable treasure. In fact the QM through its numerous techniques makes the work of the company more and more organized, standardized and efficient [1].

In order to get the most of these QM theories, and to win the trust of their clients, companies tend to follow certain standards and recommended specifications that have proven their efficiency in the business world. For instance the Capability Maturity Model Integration CMMI and The International Standard for Quality management ISO 9001: 2008, which is one of the best recognized QM regimes, and probably the most widely implemented worldwide since it deals with quality and sustainability and their integration in the company [2].

But despite the fact that these theories are normaly beneficial to every company in an equal way, some companies are still more advantageous than the others. And that's due to the fact that they tend to automatize most of the functionalities of the QM.

In the realm of this very hot topic, our graduation project comes as a perfect fit with this context. Its topic is the design and the development of an application, in which we will try to improve and automatize some functionalities of the QM within Focus, the company in which the internship took place.

Our work will be presented throughout this report, accordingly to the following plan. We will start by a brief introduction of the hosting company: its history, present and future, in the chapter "Project's frame", and we will put our project in its correspondent framework, and introduce the adopted work methodology. In the next chapter "Preliminary Study" we will shed

the light on some of the most important keywords of our work and a brief literature review in the “state of the art” section, and then we will try to analyze the current way of maintaining the QM around the hosting company, and we will put into question its efficiency, and then we present, defend and argue our solution. After that we will make a list of the functional and non-functional requirements related to our application, afterwords we will present them in a semi-formal way by using a set of use case diagrams, in the chapter “Requirements analysis and specification”. In the next chapter “Design and structure”, we will go through the application’s architecture and how it was designed. Afterwords we will finish by giving a glimpse of the different technologies that were used to make the project, and presenting and displaying the main interfaces in the “Achievements” chapter, illustrated by a set of screen shots, and presenting the timeline followed throughout our project.

Chapter 1

Project's frame

THroughout this chapter we will present Focus, the company in which our project was held. And then we will proceed to put our project in its general framework, by giving the reasons behind elaborating it and presenting the expected work to do. And finally we will put in evidence the adopted methodology of the work.

1.1 Presentation of the hosting Company

In this part we shed the light on Focus, the hosting company, and give a brief introduction to its history and then we give a glimpse of its management department.

1.1.1 Generalities and mission of Focus

Focus *(figure 1.1)* is a Tunisia-based Software Development and Information Technology (IT) Services Company founded in 2003.



Figure 1.1: The logo of Focus

It delivers cost effective outsourcing software and IT solutions based on advanced technical skills, IT expertise, and strong commitment to quality assurance.

Working with customers around the world, Focus offers significant experience and expertise in a broad range of technologies, with a portfolio of value-added IT services that includes:

- Software engineering and development.
- Software Maintenance and Support.
- Software Testing and Validation.
- Application Customer Technical Assistance Services.

Thanks to Focus's dedication and strong determination to deliver the best software services and outsourcing solutions to its clients, and the experienced dedicated staff and IT engineers with strong team spirit and success commitment, the company has grown and reached a size of more than 200 employees by the end of 2011 with revenue of more than 5.6 million USD[N1].

1.1.2 History of Focus

Focus was founded in 2003 with its first achievement of assuring high quality development services to its first partner SIEMENS AG [N2]. The major milestones for the company are:

2003 - Creation of Focus and partnership with Siemens Com Tunisia

2006 - Partnership with Continental Automotive (former Siemens VDO)

2006 - Partnership with Nokia-Siemens Networks (Telco, former Siemens I and C)

2007 - ISO 9001:2000 Certification (AFAQ France)

2008 - Partnership agreement with SAP and Business Object

2009 - CMMi L3 certification

2010 - ISO9001:2008

2011 - Partnership with Parrot

2011 - Strengthening the Partnership with SAP via new agreement for Software Development services

2013 - Creation of our subsidiaries in France and Germany

2013 - Launch of the IT Infrastructure Services

1.1.3 The Quality Management in Focus

The Company is focusing on project-level quality systems, to ensure that every customer engagement progresses smoothly [N3]. The Project Management capabilities are enhanced and driven by:

- competency development through extensive trainings and certifications:
  - In March 2007, Focus obtained its AFAQ certification ISO 9001-version 2000 and carried out intermediate assessment in order to upgrade the actual Quality Management System.
  - In December 2009, Focus carried out successfully a CMMi L3 SCAMPI A assessment and reached the CMMi Maturity Level 3. Focus has been reappraised as fully compliant at Maturity Level 3 in December 2012.
  - Since March 2013, Focus renewed its ISO 9001-version 2008 certification by AFAQ AFNOR INTERNATIONAL. AFAQ certificate No. QUAL/2007/28885 valid until 20/03/2016.
- Infrastructure development through integrated project management tools The following activities are certified:
  - Design, Implementation, and Integration of applications, Third-party applications maintenance, third party applications test and validation
  - Technical Support of third party applications

1.2 Project overview

In this part of the chapter we present, as a first moment, the dilemma that caused the responsible of the QM department to propose this project. And as a second moment, we will give the goal of our work. And we will finish by presenting briefly the methodology chosen for our work.

1.2.1 The reasons behind the project

As we mentioned, our hosting company Focus is an IT-outsourcing-based company, thus the maintaining of a good quality is a matter of survival.

And ever since, Focus as the presentation section bellow showed us, was certified ISO 9001:2008 and CMMI which are among the most recognized QM standards [1], it has never stopped aiming for having even more efficient measurements to enhance its Quality Management System (QMS). Nevertheless, the hand-based work to manage the quality management in the company and to keep up-to-date information about the evolution of the metrics of quality can turn out to be an overwhelming and very hard mission, especially when the amount of documents increases by time.

1.2.2 The Aim of Our Work

The goal of our application is optimizing the Quality Management System in Focus. Actually, we will be expected in this project to design and develop an application that would enhance some of the company's QMS functionalities, namely the management of quality metrics (or Key Performance Indicators: KPI), which drives and controle all the aspects of quality in the company, and the management of the documents that are generated during the documentation of quality processes, and the automatization of the audit management process.

This application, hosted in the company's servers, should make it work in an efficient way, by making the management of documents and the access to the KPIs a lot more easier, and presenting the relevant data with clear charts by automatizing the whole process. It should also help the responsible in the process of the audit. This application must be based on the J2EE architecture and more precisely with the Spring framework, with a collaboration of the Hibernate framework for persistence matters. It must also interact with a directory to extract the list of users, in this case: Lightweight Directory Access Protocol (LDAP) . We should also follow the Rational Unified Process (RUP) methodology specifications during the realization of the project, which will be detailed in the next section.

1.2.3 The adopted work methodology

In order to meet the responsible of the QM department's desire, we would rather work accordingly to the Rational Unified Process, annotated RUP, methodology specifications.

This method is specifically one of the most recognized implementations of the unified process (UP). It gives a development environment that respond to the fundamental needs of the designer and the client [11] [13].

The Unified Process (UP) is a use-case-driven, architecture-centric, iterative and incremen-

tal development process framework. The UP is broadly applicable to different types of software systems, including small-scale and large-scale projects having various degrees of managerial and technical complexity, across different application domains and organizational cultures.

Unified Process is considered an agile methodology, thus it embraces the idea of collaborative and incremental iterative development.
Instead of other methodologies which focus solely on engineering disciplines or project management disciplines, RUP is a complete methodology focusing on engineering and support disciplines. It is important to say that RUP is a development process, not software process [13]. It has no references to phases such as production and maintenance.

Since RUP is nothing but an implementation of the UP methodology, it is defined by the following concepts:

- Iterative development, which means the rework of the scheduling strategy where the time is set aside to revise and improve parts of the system. Being iterative, it helps to improve the product.
- Incremental development, which is a scheduling strategy in which the various parts of the system are developed at different times or rates, and integrated as they are completed. Being incremental helps to improve the process.
- Continuous integration, which means the practice where developers integrate their development frequently, usually through a tool which performs unit tests in a daily basis to detect errors as soon as possible.

Conclusion

In this first chapter of our report, we presented the hosting company and we gave a glimpse of its history, present and future. We tried also to make it clearer what our project was needed for, and how we should proceed to develop this application by the "working methodology" part. In the next chapter we will go further into the analysis of how currently Focus does its QM.

Chapter 2

Preliminary study

In this chapter we will start by a study of the state of the art, in which we present, the most important key words of QM, and a brief literature review. Afterwords we will analyze how Focus, namely its quality management department, manages to maintain the QM. Then we will put into question its feasibility and efficiency by shedding the light on the dilemma, and we will finish by presenting and arguing our solution.

2.1 Study of the state of the art

Here we will try to answer the question: "what is the importance of quality in the growth of the IT-Outsourcing-based companies and how it can be enhanced?" Therefore, we will start by a brief investigation of the quality and computerization. Then we will provide a set of definition of the most relevent keywords of the quality. Not only because they will help to better understand the project, but also they will be the basis to understand the section “Assessment of the current situation in the company” in which we will display how the company manages to keep its QM well maintained.

2.1.1 literature review of quality in outsourcing

In this section we investigate the importance of the quality in outsourcing, since our hosting company is an outsourcing-based company, and why managers tend to digitalize the management of the quality. This will be through a brief literature review.

2.1.1.1 Importance of the quality in the outsourcing

Companies which are specialized, fully or partially, in the IT-outsourcing for functions ranging from infrastructure to software development, maintenance and support, are striving to get their share of the market, thus to survive in a highly competetive market.

Even though the definition of the success of the outsourcing experience is a bit confusing [3] [4], but as the study held by Colleen Schwarz shows, quality is among the very important basis of this venture [5].
In fact companies that need to outsource their work are getting threatened by failure and demolition. Actually one PricewaterhouseCoopers [6] report found that 55 percent of clients reported that they were not realizing the benefits they had expected from outsourcing. So they become picky of their collaborative companies. And their choice is based on the quality. And the traditionnal way of managing the quality is no longer efficient.

In fact the concept of quality does not concern only the quality of the final product, but it deals with the whole process of producing it. And it certainly deals with all the aspects of outsourcing, not only the auditing and surveillance; otherwise there will be a high defect rate, late deliveries, poor service and customer dissatisfaction issues.

To provide acceptable quality, company, especially the Small and Medium Enterprises (SME), follow certain Software Process Improvement (SPI), Capability maturity model integration (CMMI) for instance.

2.1.1.2 Computerizing the adopted quality management system

To get gain the trust of their clients, quality become very essential. But a new challenge emmerged: how to manage quality. In fact it takes too much work and effort from the responsible. For instance CMMI encourages that every phase of the product development should be documented, thus there should be an efficient way to name these documents. And the audit planning sometimes gets really complicated since there may exist conflicts in the dates and the audetees.

So to efficiently manage the QMS, companies computerize many tasks. Sometimes they build even their own information system to maintain the excellence[17]. So the real challenge for IT-Outsourcing-based companies now is very much about implementing efficient programs, applications, information systems in order to survive. Adopted solutions can be as an automatization tool for repetetive tasks like audit planning or naming the generated documents. Or

to represent the information in a meaningful way through charts and dashboards.

2.1.2 The project's Key words

Some of the most important definitions that are important for our project are listed below.

2.1.2.0.1 Quality Management (QM). The QM by definition ensures that an organization, product or service is consistent [2]. It is based on four main components: quality planning, quality control, quality assurance and quality improvement. Quality management is focused not only on product and service quality, but also the means to achieve it. The QM is guided by a set of KPIs (Key Performance Indicator). And often there should be audit for the existing processes.

2.1.2.0.2 Quality Management System (QMS). Quality management systems [2] are the set of business processes or "structured procedures" that enables the realization of the company's quality policy and quality objectives, with granting for each responsible a particular task in a hierarchical way. The documentation is an important way to keep track of the achieved procedures.

2.1.2.0.3 Reference generation. Documentation plays an important role in ensuring that the tests and checks have been carried out as planned and the results accurately recorded and forwarded to the specified authority [2]. They must be referenced by a unique reference.

2.1.2.0.4 Quality Policy. The quality policy is a measurement that is taken to set the aim of the firm or the company in a particular time [2], a phase in the company. This policy is set by the "top management" and then announced to the other workers.

2.1.2.0.5 Quality Audit. The Audit is a periodic, independent, and documented examination and verification of activities, records, processes, and other elements of a quality system to determine their conformity with the requirements of a quality standard such as ISO 9000 [12][2]. Any failure in their proper implementation may be published publicly and may lead to a revocation of quality certification also called conformity assessment or quality system audit.

2.1.2.0.6 Process. Sequence of interdependent and linked procedures [16] which, at every stage, consume one or more resources (employee time, energy, machines, money) to convert inputs (data, material, parts, etc.) into outputs. These outputs then serve as inputs for the next stage until a known goal or end result is reached. Every company has various processes to keep their work efficient. Each process has one or many KPIs to measure its development.

2.1.2.0.7 Software Process Improvement (SPI). A Software Process Improvement is a way adopted by companies to improve their quality of service or products. For instance we find the Capability Maturiy Modal Integration (CMMI)[14]. CMMI is a process improvement maturity model for the development of products and services. It consists of set of the best practices that help companies to enhance their development and maintenance activities of a product lifecycle from conception through delivery and maintenance.

2.1.2.0.8 Process Area. Is a set of processes and practices which concern particular important areas for a company, and which aim at improving these areas, CMMI provides 22 Process areas (PA). [14]

2.1.2.0.9 Key Performance Indicator (KPI). The quality management is based in a big portion of it on using indicators to follow and sometimes predict the development of a certain important factor in the company [15] . Those indicators are called Key Performance Indicators (KPI). Every KPI Is related to a process, a project, a responsible… The KPIs are quantifiable measurements, agreed to beforehand, that reflect the critical success factors of an organization. They will differ depending on the organization. For example, a school may focus its Key Performance Indicators on graduation rates of its students. Whatever Key Performance Indicators are selected, they must reflect the organization's goals, they must be key to its success, and they must be quantifiable (measurable). Key Performance Indicators usually are long-term considerations. The definition of what they are and how they are measured do not change often. The goals for a particular Key Performance Indicator may change as the organization's goals change, or as it gets closer to achieving a goal.

2.1.2.0.10 Dashboard. Management is a tool that sums up the business's status in one glance. It presents a number of KPIs in a visual display Dashboard [13]. The data for these indicators is pulled from a variety of sources: departmental databases, customer relationship

management systems, staff reports and web stats...



Figure 2.1: Example of dashboard from Oracle Corporation

The presentation is done through graphical *(figure 2.1)* [13], tabular and textual means, and usually allows drill-down into specific measures.

2.1.2.0.11 Outsourcing. The outsourcing [5] is the fact that a company shifts entierly or partially organizational activities. The aim of this act is to provide a cheaper product, and to have more productivity. The outsourcing may concerns the services that are inside the company or Information Technology (IT) outsourcing.

2.2 Assessment of the current working method in Focus

In this part we focus on explaining the most relevant tasks that, currently, the responsible of the quality management is supposed to do in our particular case: the company Focus. These

task, after a deep study of the work's process, were divided into three main tasks. We present them one at a time next.

2.2.1 KPI management and tracking

In order to supervise and manage the KPIs, currently, the responsible should spend most of his time looking through tons of hard copy, hand-managed documents: excel, word, pdf…

Since the responsible is ought to make his decisions based on the information within these documents, he has to bare the trouble of knowing and recognizing each one of the project's details *(figure 2.2)* namely:

- The "frequency": how often the KPI is calculated and/or updated (every semester-year-month- trimester)
- The "formula": how the KPI values are calculated every period of time
- The "target value": the desired value that a KPI should attain, it is a yearly value.
- The "action launching point": the starting value for every period.
- The "applicable projects": the projects that are evaluated by the KPI
- The "state of deployment": indicates whether the KPI is deployed or not
- The "related process": under which process this KPI should be
- The "process area": to which process area this KPI is related
- The "name of the process": to which the KPI is attached
- The "responsible"
- "How to collect" and "analyze"

As a concrete example we took a snapshot of the actual excel document, illustrated by the figure *(figure 2.3)*, used by the QM responsible to maintain the information of the KPI, and to kept up-to-date the KPIs values accordingly to their appropriate frequency. In this example we recognize the three KPIs "Sales turnover", "number of new agreements" and "customer satisfaction". Those KPIs are under the same process "Marketing and sales". Every KPI has its own formula, responsible target and frequency.

Marketing & Sales				
METRIC: SALES TURNOVER				
FORMULA	**Target**	**Action launching point**	**Applicable Projects**	**Deployment**
[sales turnover increases]% = [CA(n+1)-Ca(n)]/CA(n)*100	Half yearly increase of 12,5% Yearly Increase of 25%	Half yearly less than 10% Yearly less than 20%	–	**deployed**
METRIC: NUMBER OF NEW AGREEMENTS				
FORMULA	**Target**	**Action launching point**	**Applicable Projects**	**Deployment**
Number of new agreement during the last year	**1**	**0**	–	**deployed**
METRIC: CUSTOMER SATISFACTION				

Figure 2.2: Information related to the KPIs management

This figure has all the attributes of the KPI and the next includes its values.

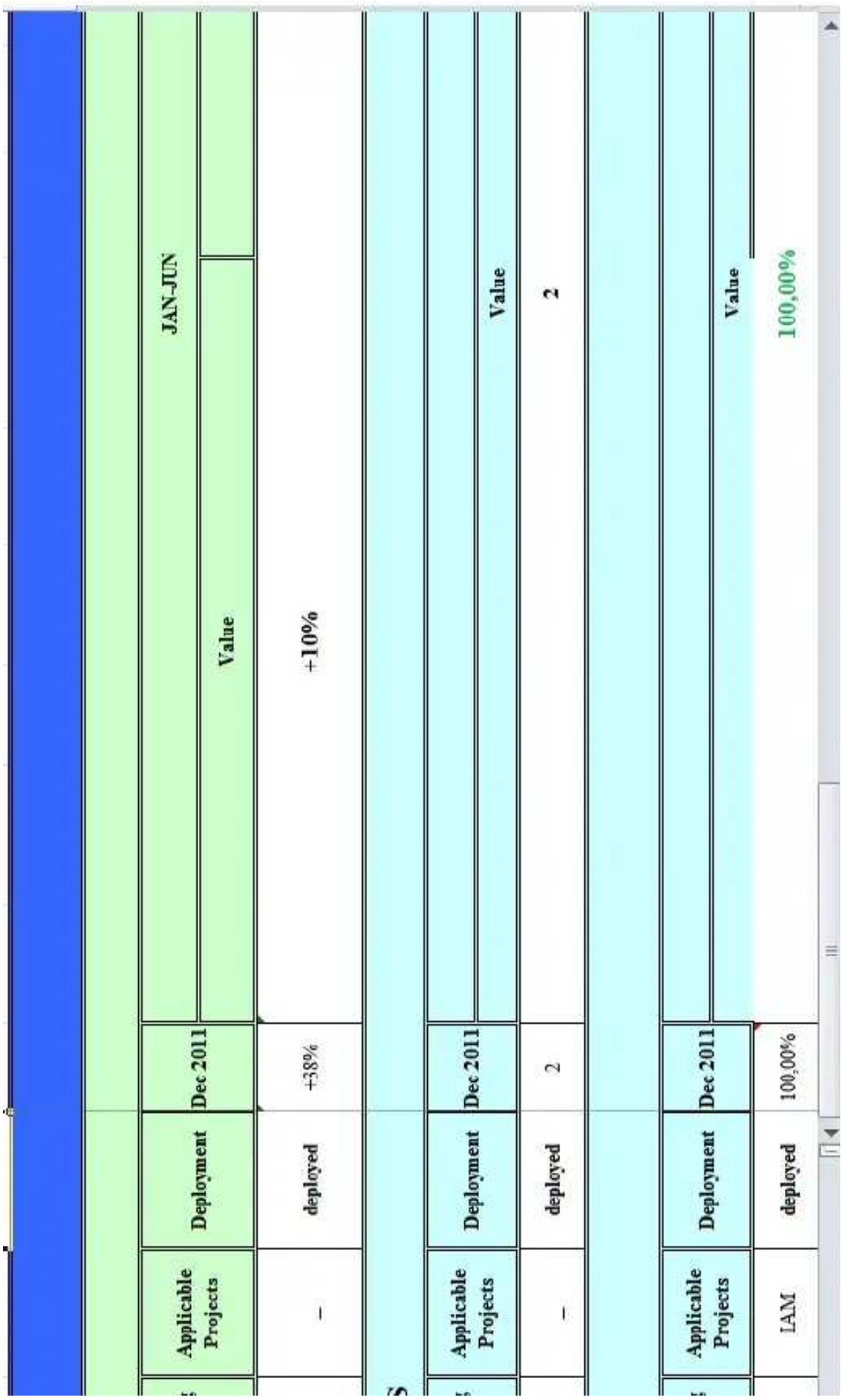


Figure 2.3: Update of a values for a KPI

These KPIs are managed by a referenceing system explained next.

2.2.2 References generation

The challenge of the QM responsible for this particular task lies behind following the strict company's "naming rules" to keep the documents traceable. The figure *(figure 2.4)* provides a concreate example of references used in the naming process in Focus. These references are based on:

- The name of the document
- The extension of the document
- The related process
- The version of the document

	DOCUMENT	REFERENCE	Reception Date
3	TR0145_Plan de Validation_- .dotx	ED/ARD/001/E	
4	Plan de validation.doc	ED/ARD/002/E	
5	FLC_Plan de Validation_ACU450_V1.1.doc	ED/ARD/003/E	
6	FLC_Drivers_ACTIA_SODIELEC_2013-05-16.docx	ED/ARD/004/E	
7	FLC_Plan de Validation_ACU450_V1 1_Retour_Focus.docx	ED/ARD/005/E	
8	FLC_Dirvers_ACTIA_SODIELEC_2013-04-23.docx	ED/ARD/006/E	
9	DEV-P004189-130122-1 V01 Sp log ACU450.pdf	ED/ARD/007/E	
10	ACS450 M&C Interface revl.pdf	ED/ARD/008/E	
11	20130423_CR_Avancement_Drivers Sodielec.pdf	ED/ARD/009/E	
12	DEV-TRL_1S_TRL_S.pdf	ED/ARD/010/E	
13	47_TRL_1S_CG-1124RevE_.pdf	ED/ARD/011/E	
14	48_TRL-2S.pdf	ED/ARD/012/E	
15			
16			
17			
18			

ABREVIATION | GENERAL ED | ARDIA ED | CMMI ED | SCL ED | NPG ED | Firma ED | ATDT ED | TUN (WS)

Figure 2.4: sample of a referencing document

This latter figure shows how the responsible of QM does the references by hand with Excel file.

2.2.3 Audit planning

Another cornerstone of the QM in companies, specifically Focus, is the audit planning. How the audit planning is held in the company is shown by the MS-word document of the figure *(figure 2.5)*

date. : 25/01/2013	JANUARY & FEBRUARY 2013 AUDIT PLAN	FOCUS Page : 1

1 AUDITED PROCESSES:

The audited processes are:

- HUMAN RESOURCES MANAGEMENT PROCESS
- CHANGE MANAGEMENT PROCESS
- EXTERNAL TECHNICAL SUPPORT PROCESS
- INTERNAL SUPPORT PROCESS
- PURCHASING PROCESS
- SPECIFICATION & DEVELOPMENT PROCESS
- MARKETING AND SALES PROCESS

2 AUDIT PLAN

Date	Hours	Process	Auditor	Auditees
31/01/13 12/02/13	On site audit realization			
31/01/13	10h-11h	SPECIFICATION & DEVELOPMENT PROCESS	Smati Manel	Mohamed Amine Selmi Samir Ben Saoud Khaled Hamdi
31/01/13	11h-11h30	CHANGE MANAGEMENT PROCESS	Smati Manel	Anis Hammami
31/01/13	11h30-12h	EXTERNAL TECHNICAL SUPPORT	Smati Manel	Anis Hammami

Figure 2.5: Sample of audit planning

In fact, a list of "audited processes" is added to a list of dates, hours, auditors and audited responsible. The responsible have to plan an audit for each process in the list, and then let everyone concerned by this audit know by telling them directly or by the phone.

2.3 Critics of the current way of work

With the previous assessment, we defiantly find some tasks that cannot be classified as "easy". And we sum up the negative points of the current way of working by the following points:

- It is hard to keep track of the development of the KPIs when the responsible have to go through big amount of documents without an efficient tool to filter and display the preferred KPIs and processes.
- The huge amount of documents makes it almost impossible to manage them by the company's strict "naming roles" during the referencing phase of the document.
- Numbers are just meaningless when they are not well presented by charts. The KPIs should be well presented by diagrams and charts.
- The process of audit: planning, announcements and reporting, is quite complicated and overwhelming for a human being to manage it by himself.
- Most of the responsible tasks rely on interacting with other workers of the company, that's why he keeps referring manually to their roles and permissions via the company's light database LDAP directory deployed on the company's servers.

2.4 The proposed solution and expected work

Based on the literature review we propose as a solution for the last critics, to create an application makes the access to the information about the KPIs easier via a dashboard that enables the transformation of the raw data, within the documents, into more meaningful information through the charts. It also must automatize the audit planning and announcement process, and manages the document's referencing process. This application must integrate the Spring and Hibernate frameworks and import user's roles and credentials from an LDAP directory. It must be also extendible and easy to use.

Conclusion

In this chapter we went through a brief study of the state of the art and we presented the most important QM notions, and then we attempted to put in evidence the necessity of our application, and how the current way of working is not efficient enough. In the next chapter will be enumerating the different functionalities that are needed for our application, and we will make them more clear and formal by use case diagrams.

Chapter 3

Requirements analysis and specification

After a concrete investigation of the company's work, and having a first look at the needed functionalities to help responsible of the QM, in the previous chapter, this one is fully dedicated to present and analyze them in a more formal way. We start by presenting the functional and non-functional requirements of our application. Then we list the actors of our application with their granted permissions. Finally we formalize the identified functionalities with the help of Use Case diagrams.

3.1 Requirement Analysis

Throughout this section we identify and present the different functional and non-functional requirements that our application should have by the end of our work.

3.1.1 Functional requirements

Our application should fulfill the following functionalities, ranked by order of importance.

FR1. *Dashboard management:* Our system allows the definition of the KPIs (process relative KPIs, Team relative KPIs, KPI description, KPI formula, KPI frequency, KPI target value).

It offers also the interface to track KPIs values and analysis result in case of not achievement. Users can display KPIs historic values and compare them to past values (same period last year, last period value). Each user can choose its specific dashboard by selecting the list of KPIs to display and their graphical layout and to update them.

FR2. *Audit planning:* Our system serves to the audit planning, reporting and publishing, with integration with third party "Action Management Tool" (open source Mantis).

FR3. *Document traceability:* Our application should create automatic document's references, based on the company internal documentation process. The generated references are based on the document type, related process, related project, and related project specific phase.

FR5. *Administration of the application:* which serves to manage The users (integration with the Lightweight Directory Access Protocol LDAP) and their roles.

3.1.2 Non-functional requirements

Once our application fulfilled the functional requirements listed above, it has to respect the following measurable characteristics.

NFR1.*Compatibility*, our applications should be able to be used with external programs, such as LDAP for authentication, and Mantis for reporting.

NFR2. *Availability and performance*, our application should respond to the user's demands in a reasonable amount of time.

NFR3. *Usability and Simplicity of usage*, the graphic interface should be intuitive and simple so that the user can easily understand it and simply process the needed configuration without a previous training.

NFR4. *Maintainability*, we should use the Java Coding Rules and the best practice of J2EE, so that when other developer wants to understand or extend the code will not find a problem.

3.2 Requirement specification

This section has a double purpose, first to have a better understanding of the mentioned requirements and to present them in a semi-formal way, second, to emphasize the interactions between the actors and our application. In order to break down the complexity of these goals we will use the Use Case Diagrams.

3.2.1 Identification of the actors of the application

An actor is an abstraction of a role of actual users who is in a perpetual interaction with the application. Specifically our system's actors are classified accordingly to their roles and granted permissions in the company, and they are as follow:

- A "normal user", this profile presents the users that can create, update or delete the KPIs update their values and to set the personal settings for viewing the dashboard. This profile gives the promise to the user of generating the references for the documents, and the management of the Audit process.

- The "Administrator", who is a normal user with the same granted permissions. But what is different is that an actor with an "Administrator" profile has all kind of permission to manage the application as a whole.

3.2.2 Use case Diagrams

In this section we will present the set of use case describing our application to better understand its functionalities.

3.2.2.1 General use case (system's use case)

The following diagram *(figure 3.1)* presents our application as a black box and puts in evidence its interactions with the actors in a semi-formal way. This diagram serves to limit the system and define each actor's functionalities and its interaction.

Basicaly our system presents two types of actors, "Administrator" and "Simple user", which both inherit the same permissions as a "User" with more functionalities granted for the administrator.
As a "User" the actor can keep track of the KPIs and the document's references. He can also make the audit planning. In order to keep track of the KPIs, the user can manage them, update their values or choose the settings of the dashboard.
The "Administrator" benefits extra permission, the "administration of the application" for instance adding new users, deleting, or updating their information and credentials.
knowing the fact that the application must make the destinction between the different users and their roles, with all the mentioned use cases, the authentication is the first thing to begin with.

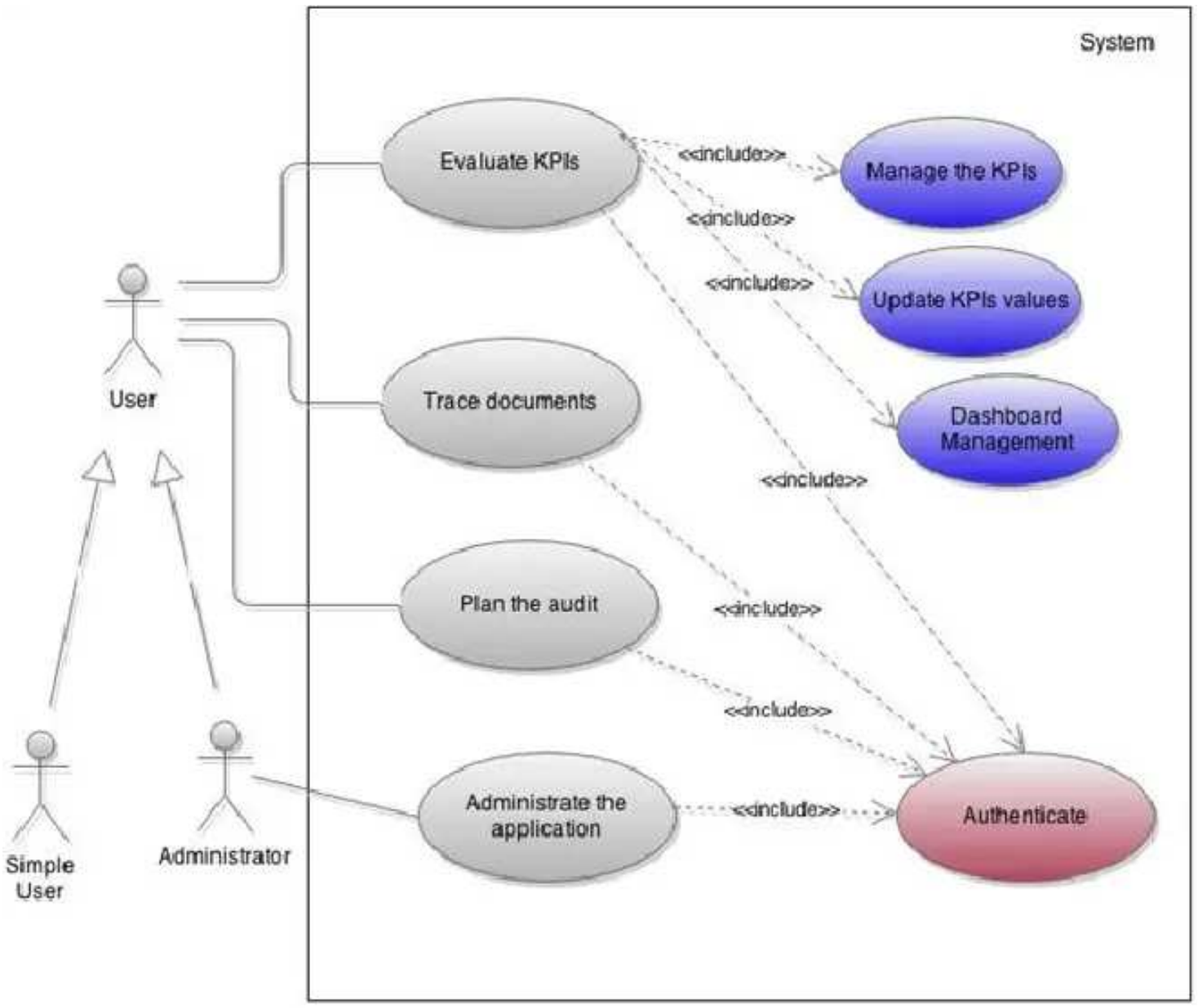


Figure 3.1: System use case

In order to attain the maximum of simplification, we didn't put all the possible use cases of our application in the previous diagram. In fact we added the next three extra diagrams to put in evidence the other use cases behind the "Manage the dashboard", the "Plan the audit" and "Manage the KPIs".

3.2.2.2 Extended "Plan the audit" use case

This use case is illustrated by the next diagram *(figure 3.2)*.

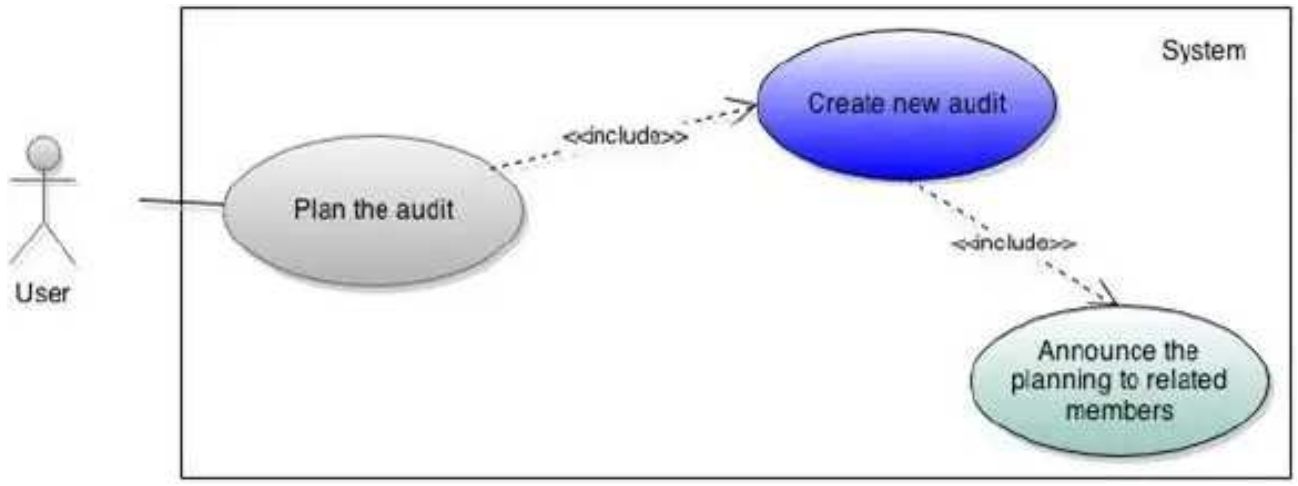


Figure 3.2: Extended "Plan Audit" use case

Actor: *the "User"*

Description: The user, both "simple" or "administrator" can plan the audit by creating a new one and then send the notification to the other concerned workers.

3.2.2.3 Extended "Manage the dashboard" use case

The different use cases behind the dashboard management are presented by the following diagram *(figure 3.3)*.

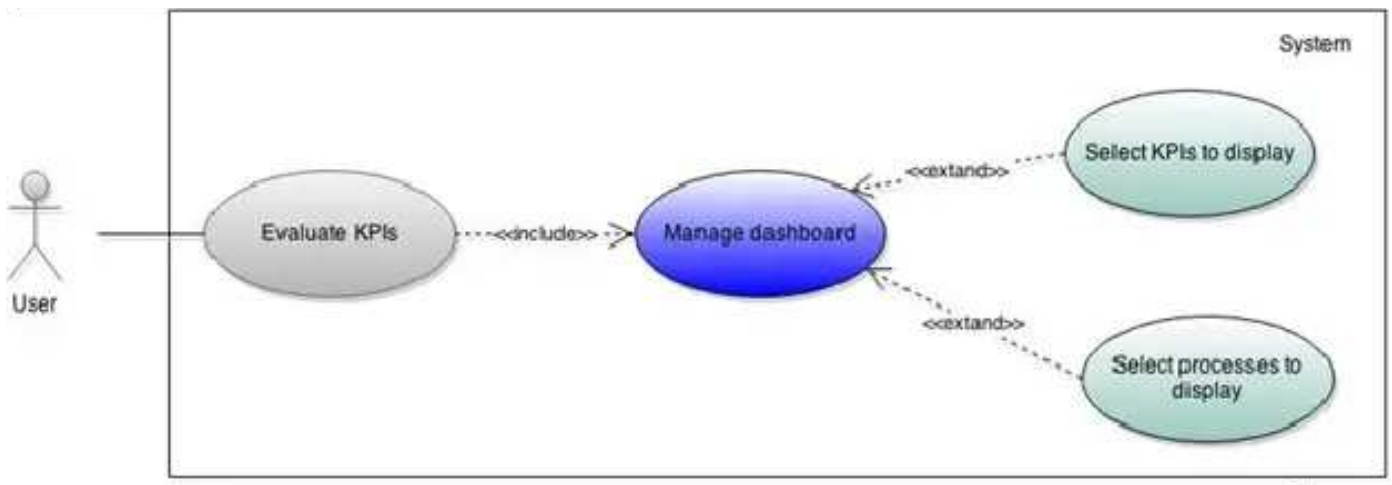


Figure 3.3: Extended "Manage the dashboard" use case

Actor: *the "User"*

Description: "User" can manage his dashboard by choosing one of the two options, whether

he selects particular KPIs to be displayed or a whole process with his related KPIs. In this way every user can define his own dashboard.

3.2.2.4 Extended "Manage the KPIs" use case

This use case is detailed by the following diagram *(figure 3.4)*.

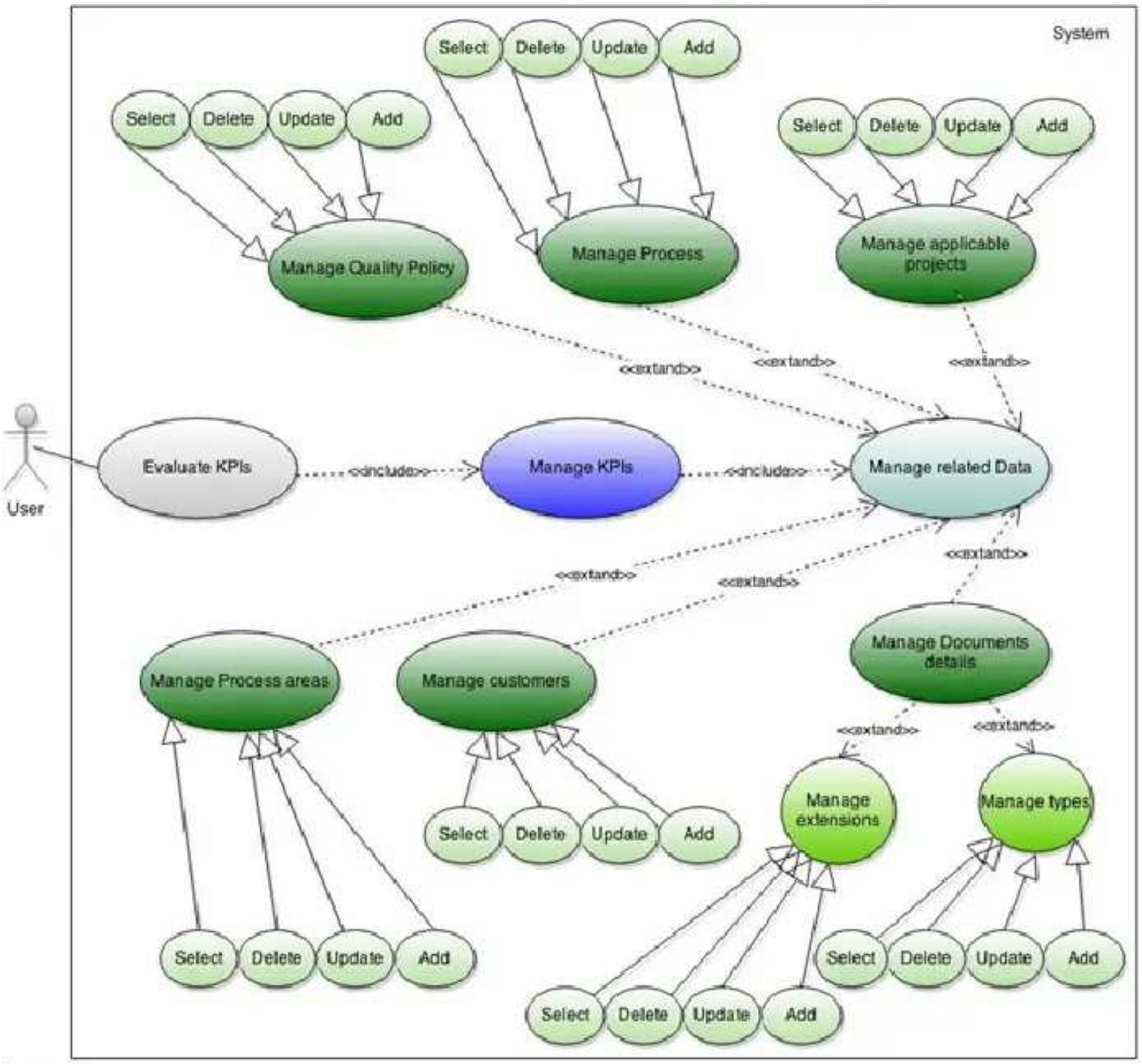


Figure 3.4: Extended "Manage the KPIs" use case

Actor: *the "User"*

Description: to manage the data related to the KPIs, the "User" may choose between:

- Manage the policy management
- Manage the Process
- Manage the Process areas
- Manage the customers
- Manage the application projects
- Manage the details of documents, which can also be extended to "Manage extensions" or "types".

And each one of the mentioned use case has the CRUD operations (Create-Read-Update-Delete) presented by inheritance relation.

3.2.3 Description of the most relevant nominal scenarios

In this section we will present the most important nominal scenarios in our application, with the help of sequence diagrams, and a brief description. Each sequence diagram considers just one profile at a time, and gives the sequence of actions between him and the system.

3.2.3.1 Nominal scenario for documents traceability

The actor provides the necessary information and they will be processed by the system. This scenario is clarified by the figure *(figure 3.5)*.

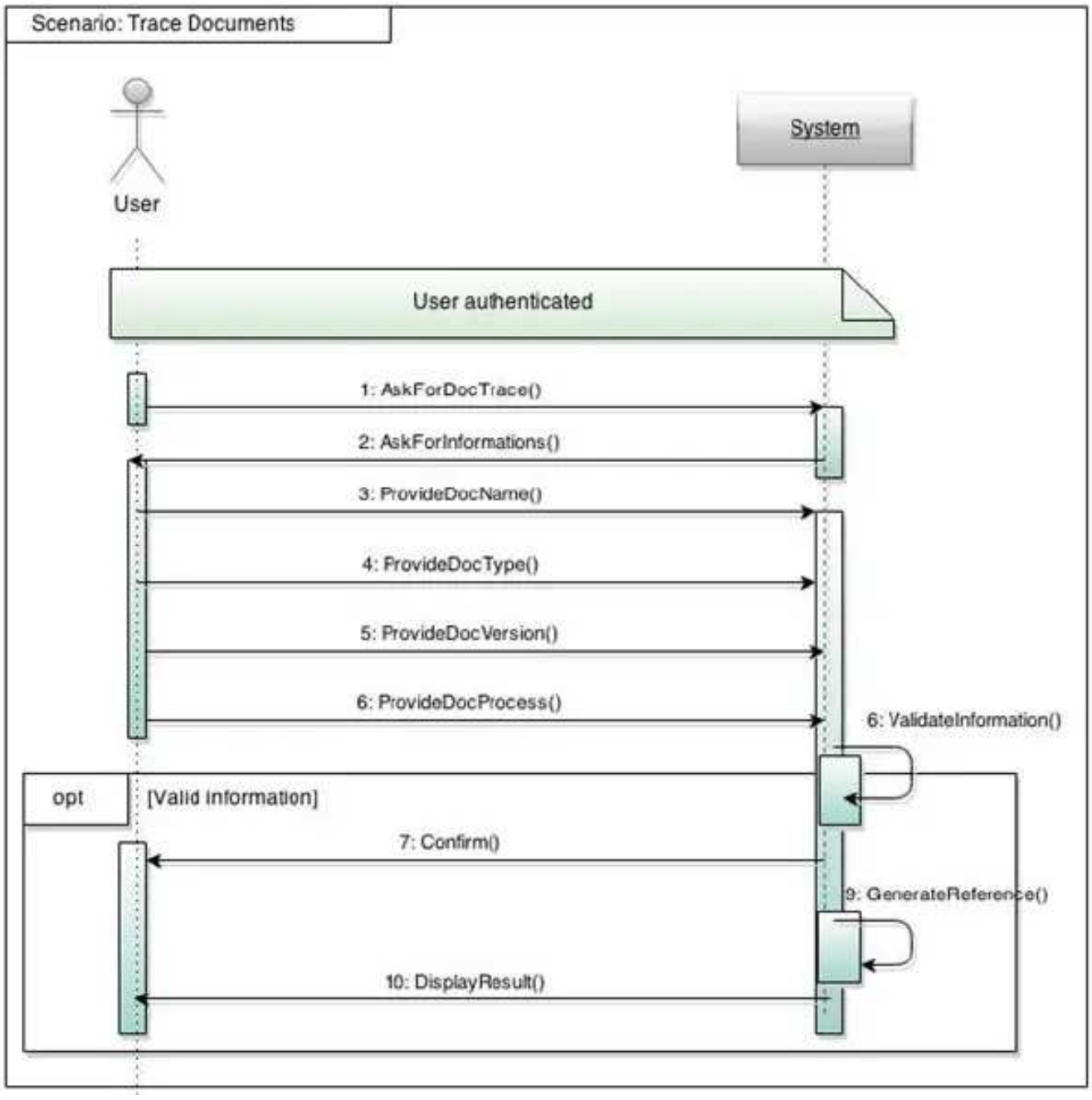


Figure 3.5: Nominal scenario for documents traceability

This figure shows that a user must be authenticated first, and then he must ask the system of reference generation. After providing the information needed for the generation: document type, name version and related process, the system process these information and then display the result for the involved users.

3.2.3.2 Nominal scenario for Audit planning

The actor provides the system with the right information, these will be processed and then a notification is sent to the related persons. This scenario is illustrated by the figure *(figure 3.6)*

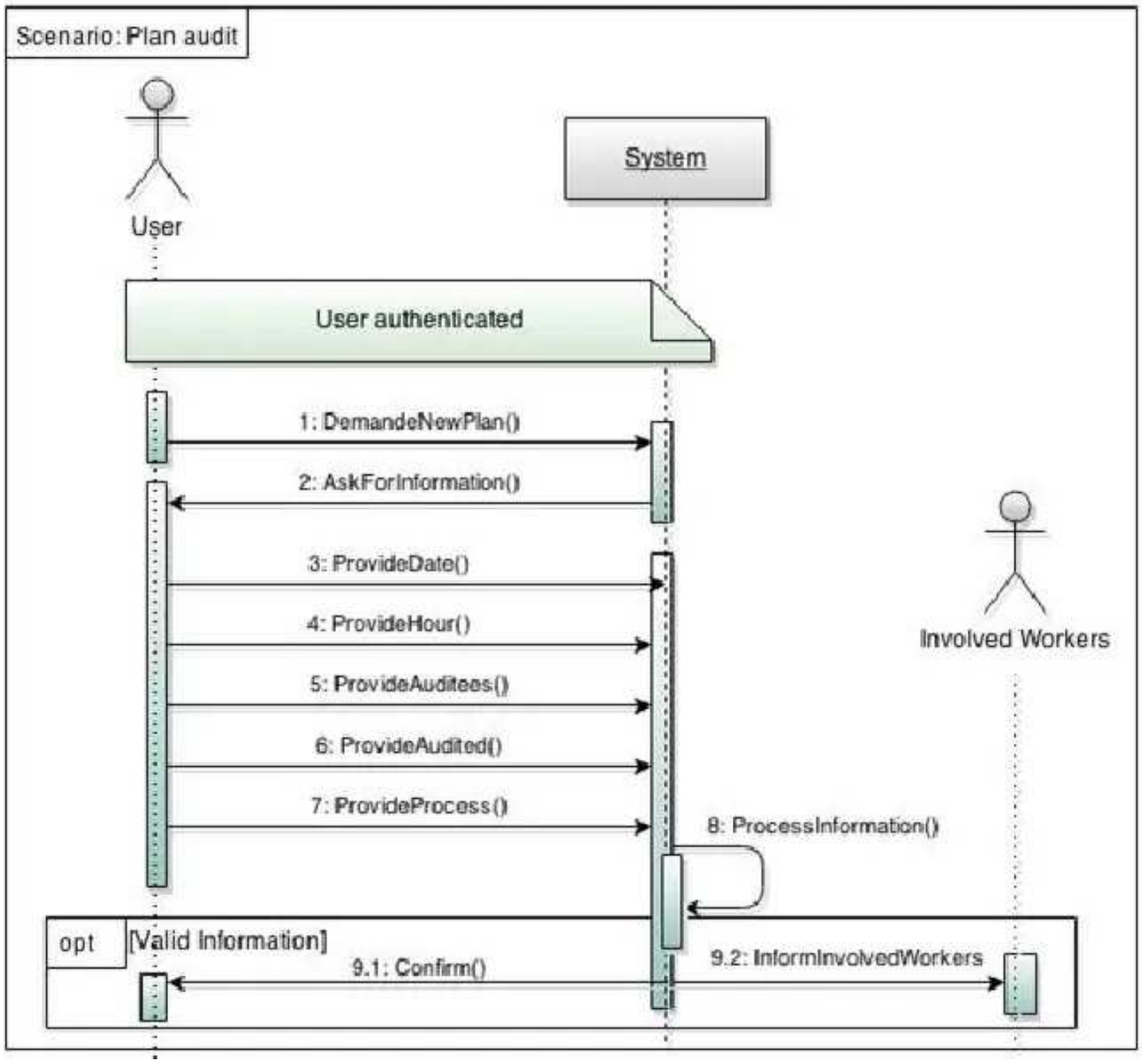


Figure 3.6: Nominal scenario for Audit planning

As usual, the user must be authenticated first, and ask for starting the audit planning. After the system validate the given information; the date, the hour, the related processes, the audetees and the audited, it confirms the user and notify the audeted workers.

3.2.3.3 Nominal scenario for KPIs tracking

This scenario is illustrated by th following figure.

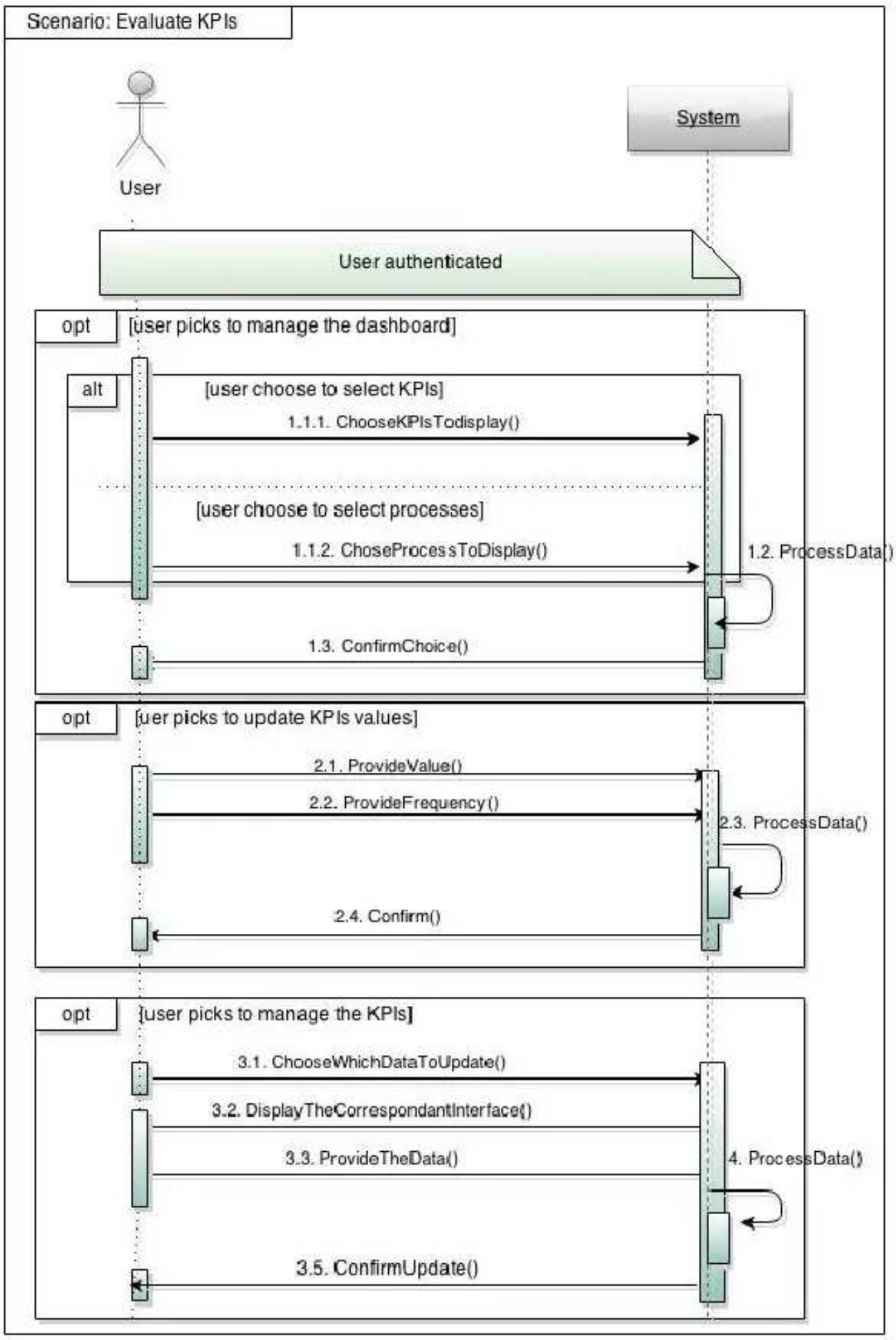


Figure 3.7: Nominal scenario for KPIs tracking

Depending on the actor's choice, and as the figure *(figure 3.7)* shows, he can

- Manage the dashboard: the actor provides the system with the preferable configuration.
- Update KPIs values: provide the exact values of KPIs
- Manage the KPIs: add-delete-update or select some related data to the KPIs

Accordingly to its choice, the user provide the information for the system. This latter process the provided information and then confirm the choice in the case of success.

Conclusion

All along this chapter we specified and analyzed the requirements that our application should deliver to the future user, and we gave the main scenarios and use case of our project. The following chapter aims to go a step further in the process of developing the application via presenting the application's design.

Chapter 4

Design and structure

HAving finished putting together the functional and non-functional requirements in the previous chapter, and giving the different use cases, we go in this chapter, deeper in the process of preparing to the development phase. This will be by choosing the suitable design for our application.

4.1 Global Design

One of the things that our project must respect is to be developed and designed around the Spring and Hibernate frameworks. This mixture is often used in the business world, and the benefits of this choice will be developed in the next chapter. But for now, in order to respect this condition we have to consider in our design process the architecture of Spring and Hibernate that is shown by the figure *(figure 4.1)* [9].

For one thing, this figure demonstrates that Spring cooperates well with Hibernate by letting the JPA (Java Persistence API) : the Spring's abstraction layer of the manipulation of the database [7], in the hands of Hibernate and the only communication between them is through interfaces. Hibernate is the responsible for the ORM (Object Relational Mapping), the correspondence between the relational world of the database and the object-oriented world of Java [8]. Meanwhile the Spring framework stays as an umbrella that embraces all the used technologies

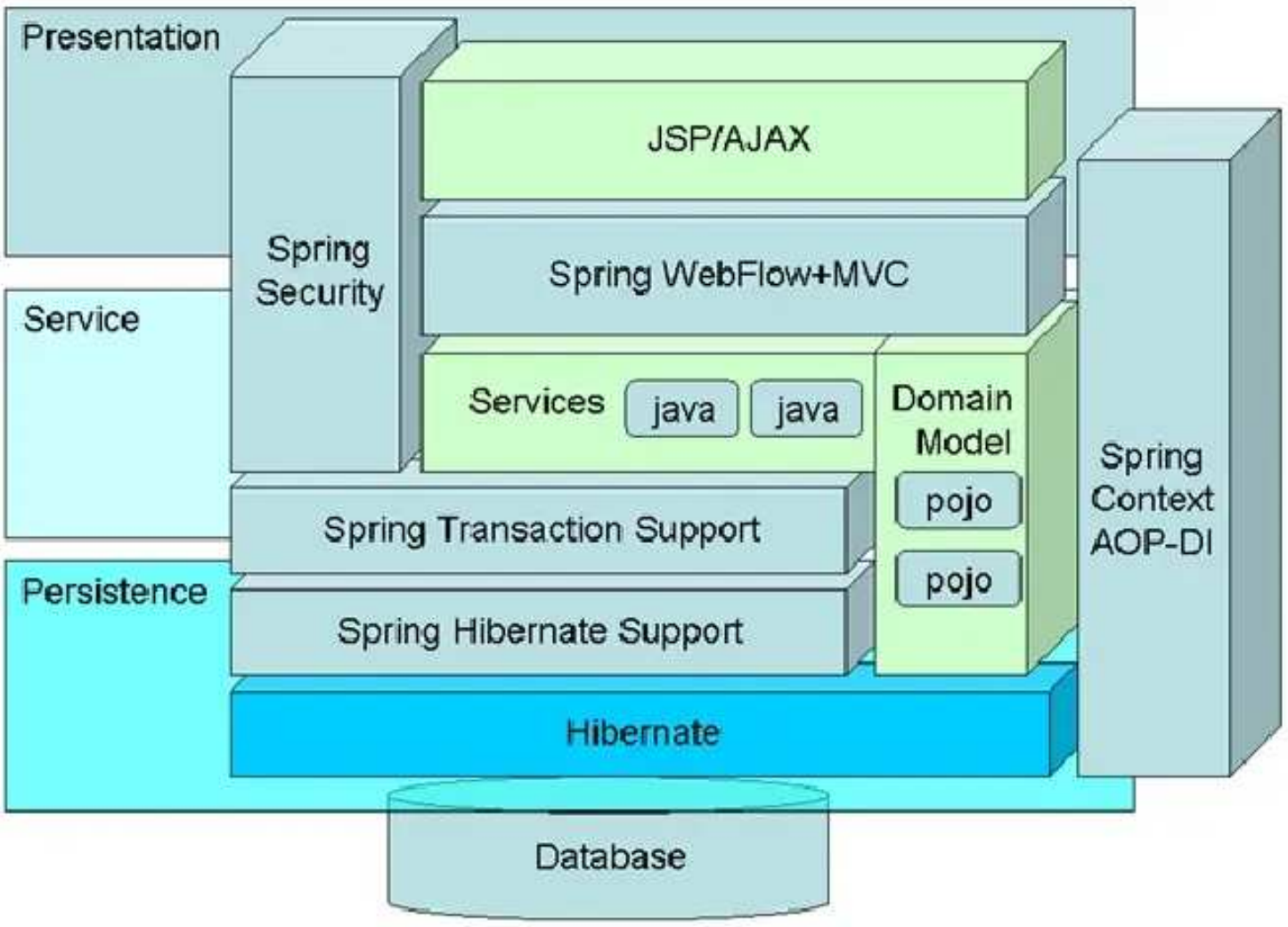


Figure 4.1: Spring and Hibernate architecture

Throughout this section we will be explaining the logical, applicative and technical architecture of our application that are based on the proposed overall-architecture: Spring-Hibernate.

4.1.1 Logical architecture

As the figure from the latter section showed us, the Spring framework maintains the clarity of the design and assures the ease of growth, through a multi-layered architecture: Presentation, Service and Persistence layers [10]. The presentation layer containes two other layers: the Controller and the View. The Model layer is another layer that in relation with all the others. All these various layers are in a constant communication. The final allure of the architecture is shown by the following figure *(figure 4.2)*.

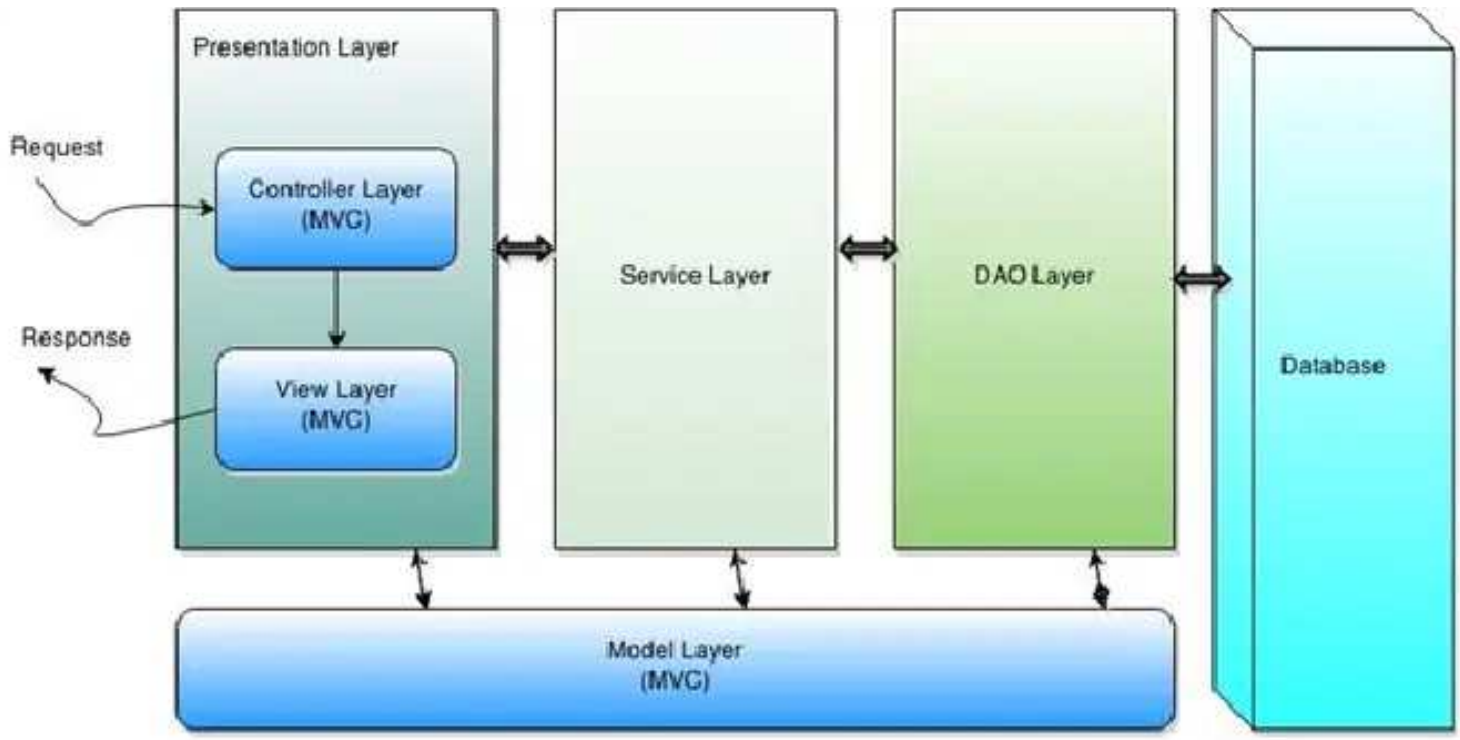


Figure 4.2: The application's logical layers

This architecture saw its genesis by mixing the three following design patterns: MVC, Spring IoC and the DAO design patterns.

- *MVC design pattern* [8] :

  The Model-View-Controller model solves inter-dependencies between data access code, business logic code and presentation code by decoupling data access, business logic, and data presentation and user interaction in order to create a flexible and loosely coupled web applications. This three layered-architecture is generally considered to be included in the presentation layer of Spring, with the exception of the model layer that can extend to the service layer and the DAO.

  - The model layer represents the data. The model does not depend on the controller or the view and in general they will consist of POJO (Plain Old Java Object) which is a Java object that does not follow the Spring framework [10].
  - The view layer displays the model data, and sends user actions (e.g. button clicks) to the controller. The view can:
    * be independent of both the model and the controller or
    * actually be the controller, and therefore depend on the model

  - The controller layer is responsible for processing user requests and building appropriate model and passes it to the view for rendering. It provides model data to the view, and interprets user actions such as button clicks. The controller depends on the view and the model.

- *DAO (Data Access Object) design pattern* [10]:
  The Data Access Object pattern helps to decouple the business logic from the database thus increasing the portability of the system. This design pattern is the reason behind the existence of the DAO (persistence) layer.
  This layer is totally inseparable from the Service layer, the main responsible of the security. In fact this layer forbids the direct access to the database and the only allowed way is through the DAO layer. It is also responsible for the transaction management. Its main concern is the business logic. It depends mainly on the model and DAO layers to do the CRUD (Create, Research, Update, Delete) operations on a persistent objects [10].

- *Spring IoC (Inversion of Control) design pattern*[8]:
  IoC is one of the techniques used to wire services or components to an application program. In Spring, the IoC principle is implemented using the Dependency Injection design pattern which leaves the system components loosely coupled and allows the developer to code to abstractions.

The previous logical layer adopts also the principle of "interface-implementation" within every layer. In fact every layer is characterized by an interface and its implementation. In this way the layers will communicate via "contracts": the interface, and ignore the complexity of its implementation. This way we can make changes in some layers without taking care of changing the higher layers.
In addition, for the DAO layer which is the one that will be in a direct interaction with the database, there will be a persistence engine that will take into account the role of persisting the objects in the database [N1]. This final cut of our architecture is clarified by the following figure *(figure 4.3)*.

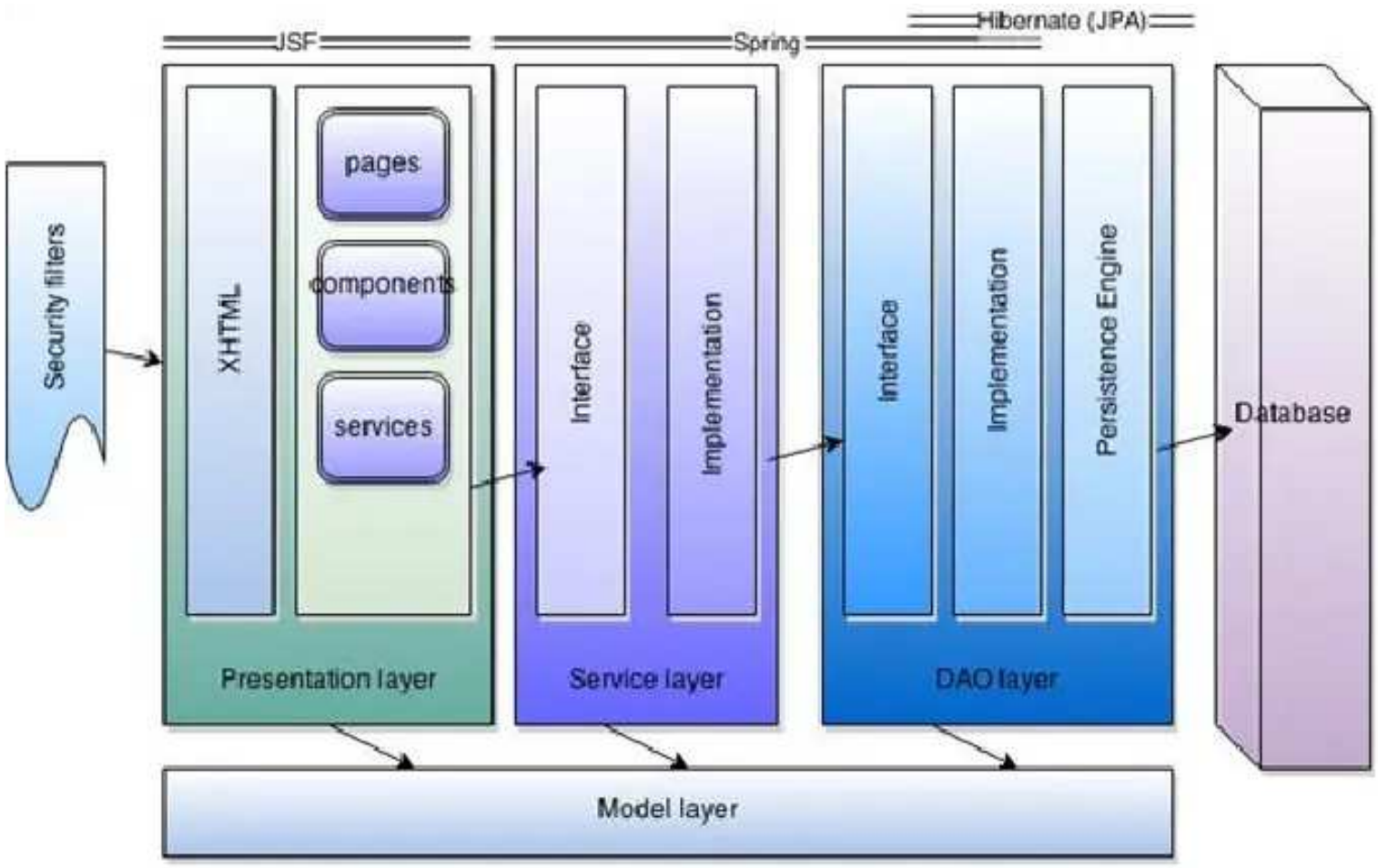


Figure 4.3: The application final design

Between the different layers, communication between the interfaces and their implementations is granted.

4.1.2 Applicative architecture

In order to stay at the same page with the previous design, and to make a concrete palpable design, we will present in this section the applicative architecture, with the help of packages diagram. In fact we made five packages *(figure 4.4)*, each one is concerned with a particular task.

- Presentation layer: this package will contain the managed bean of our application. These managed beans are the controller of the user's interfaces. They are in a direct interaction with the service layer.
- Navigation package: this package will contain the web pages that are used in the application and the navigation of the user.

- Service package: this package will contain the classes on which the business logic is built.
- The model package: this package will contain the classes that are the reflection of the database's tables.
- The DAO package is the only package that is with a direct interaction with the database.

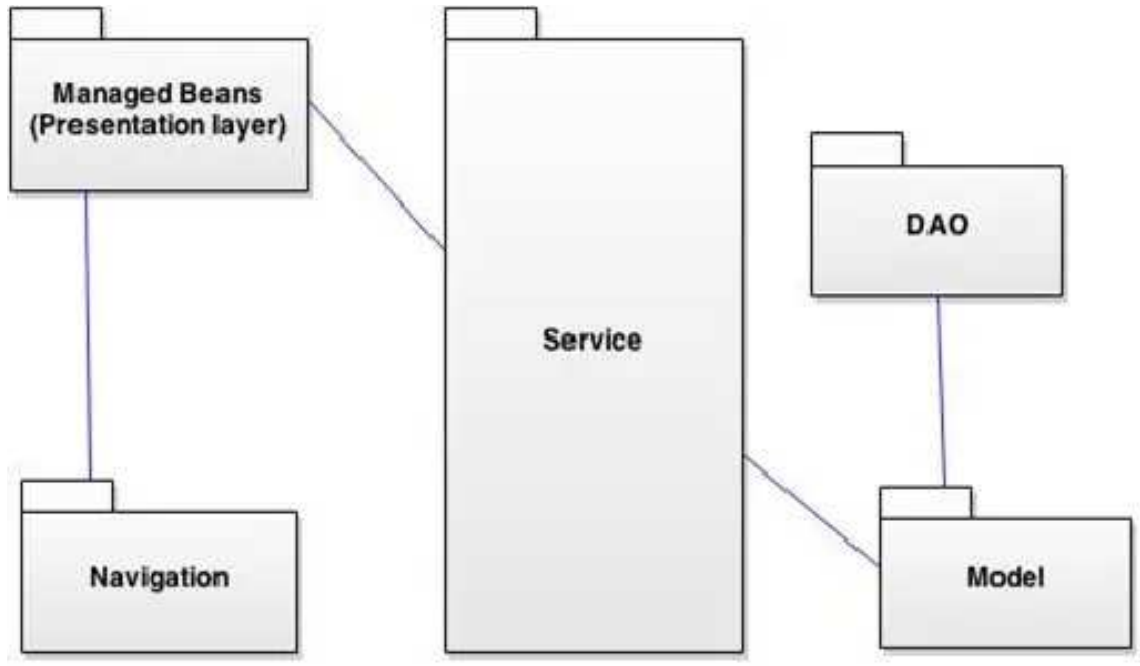


Figure 4.4: Package diagram for the application

Through this packaging strategy, we grant a loose interaction between the various components of the application. Namely between the managed beans and the navigation packages, between the service and managed bean packages, the service and model packages and the model and DAO packages.

4.1.3 Technical architecture

Until now, we did only present the logical layering design of our application which misses an important aspect: the technical design. In order to enhance our understanding of how this design is supposed to be implemented, we will present the technical architecture in this section.

Actually for one thing our application is a "web application" thus we will have to implement an architecture that considers the distributed nature of the application, a multi-tiers deployment. This particular suitable design is illustrated by the deployment diagram of the figure *(figure 4.5)*.

The client tier is the one where the client's device is. The web tier is where the web pages are living, in our case these pages are xhtml pages with integration of JSF2. The Application tier is where the application's logic lies. The data tiers is where the data is stored, in our case the users credentials and roles are stored in an LDAP server and the other information in a MySQL server.

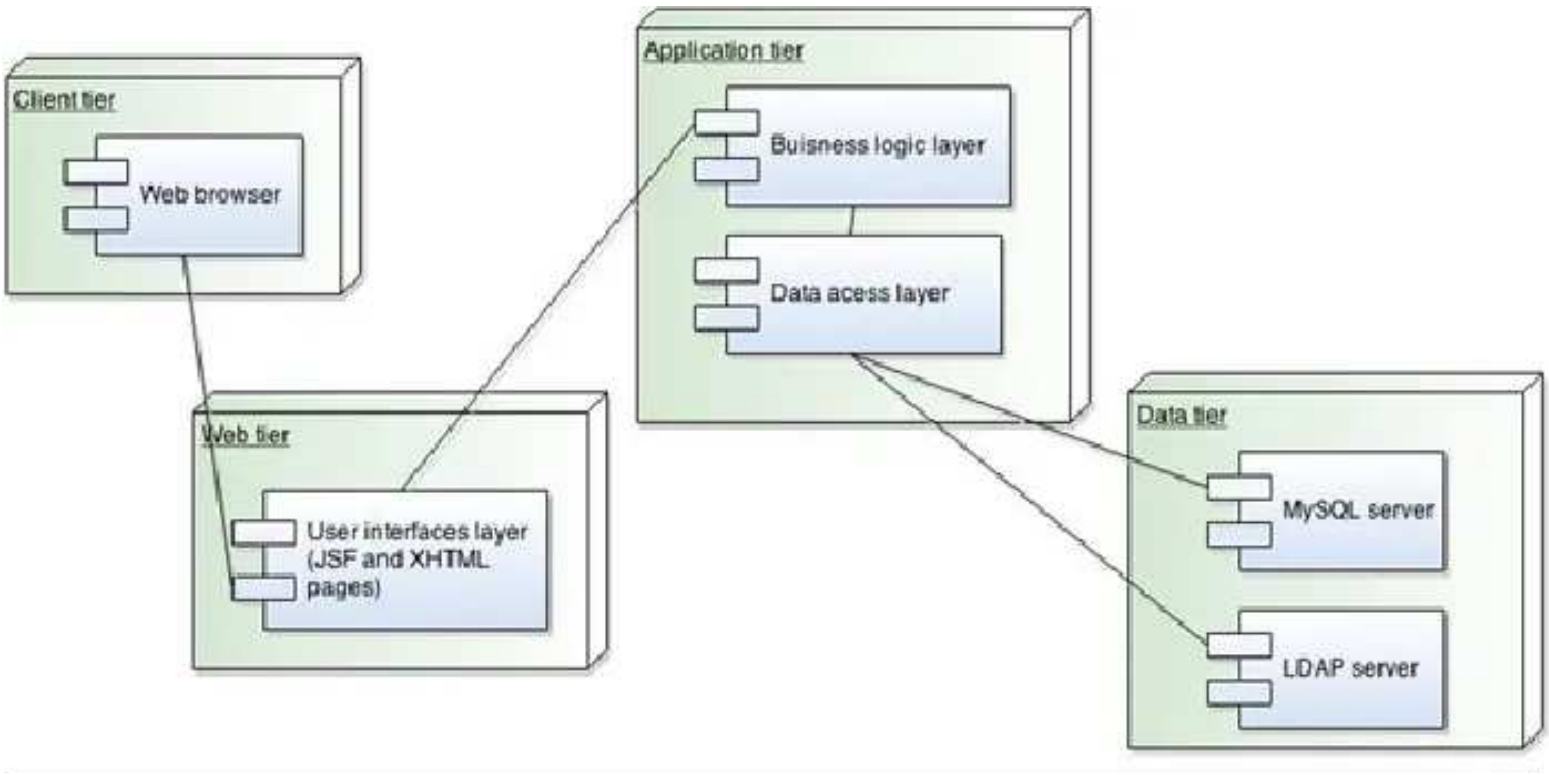


Figure 4.5: The technical design of the application

Between the different tiers communication is granted. for instance between the client tier and the user tier, the communication is provided by the http protocol.

4.2 Detailed Design

In this section we will present the specefic classes used for each paquage from the previous section.

4.2.1 Model layer (data's model)

In this part we present the detailed architecture for the model layer. It is a reflection of the existing tables in the database, for each of the database's table theire will be mapped a class with the same name. This process is called the ORM (Object Relational Mapping) that is the responsibility of Hibernate[8].

All the classes in this layer are "model", which by defenition a class that is caracterized by the following points:

- Alll the attributes are private
- For every single attribute we associate a getter and a setter
- All the methods are public
- A default constractor
- A constractor intialised with the attributes

Since the methodes will be only getters, setters and constractors we will only represent the attributes for the classes in the following figure *(figure 4.6)*. It demonstrates, the "Metrics" class is the center of the others. It is compsed by the "Applicable projects", "quality policy", "process" and "Process area", this relation is presented by an agregation. The class "Metrics Values" is a in a relation of composition with the "Metrics" class.
The "MetricValue" model/class contains the up-to-date values of the KPIs, that will be extracted and shown to the user. The other important model/class is the "Users" witch is related to the "Role" model by a Many-To-Many link. This is because one user can be assigned to many roles and vice-versa. Each one of these model is presented by a table in the database, with the same name.

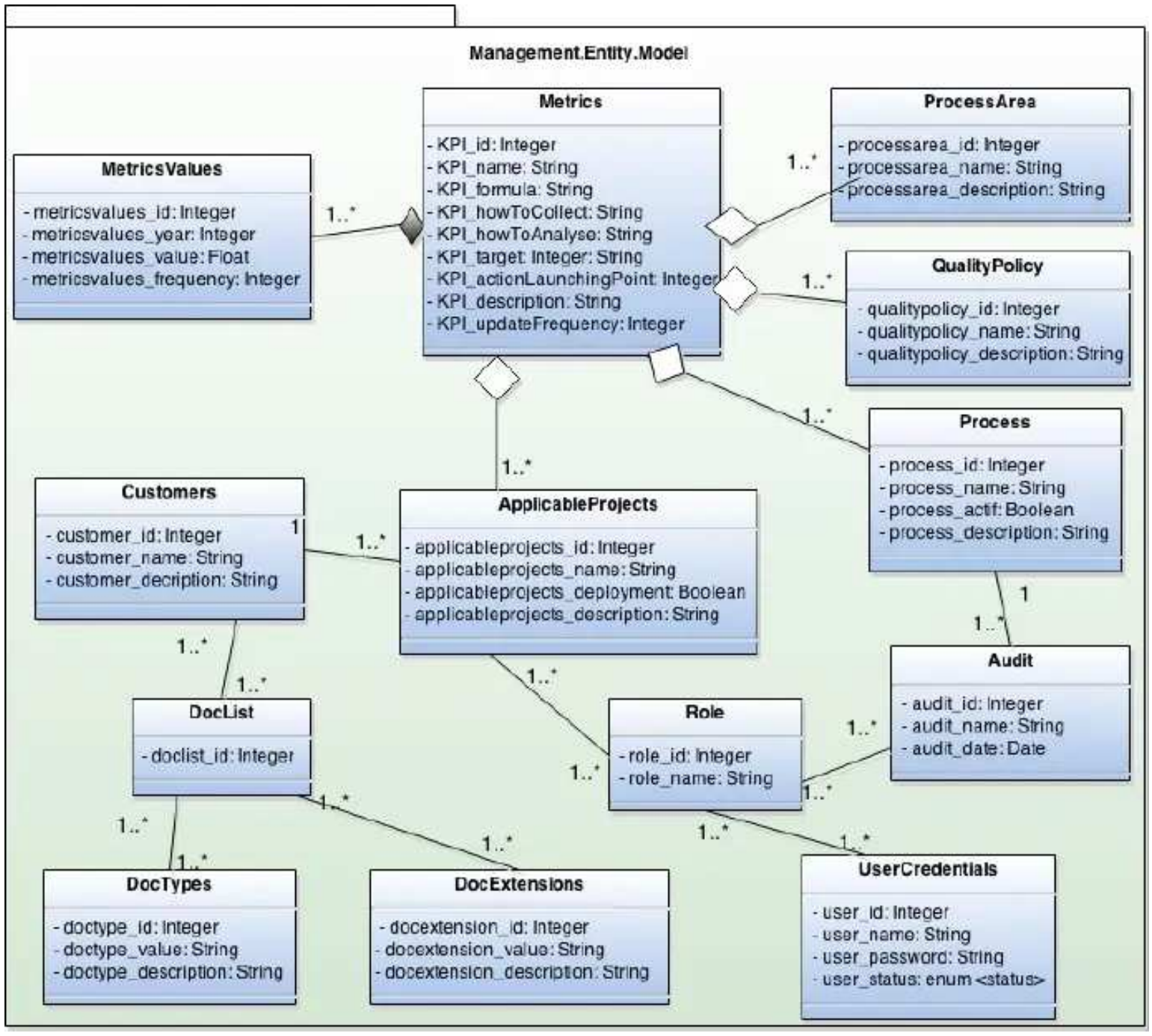


Figure 4.6: Class diagram for the Model layer

These model classes are in a constant communication with other classes from external packages. This communication is assured via interfacs. These interfaces are defined as contracts of interactions between the different layers. In fact these classes are managed and handeled by the set of classes within the DAO layer, explained next.

4.2.2 Data access object (DAO) layer

This layer is capable of managing the models and the datatable access via a persistance engine, which as mentioned before, is charged of mapping the tables with objects. This layer is devided to interfaces and implementations which present the CRUD operation on the database. Every class mentioned in the next figure *(figure 4.7)* is an interface that is implemented by a class with the same name. To make the diagram clear we will not put the implementation classes.

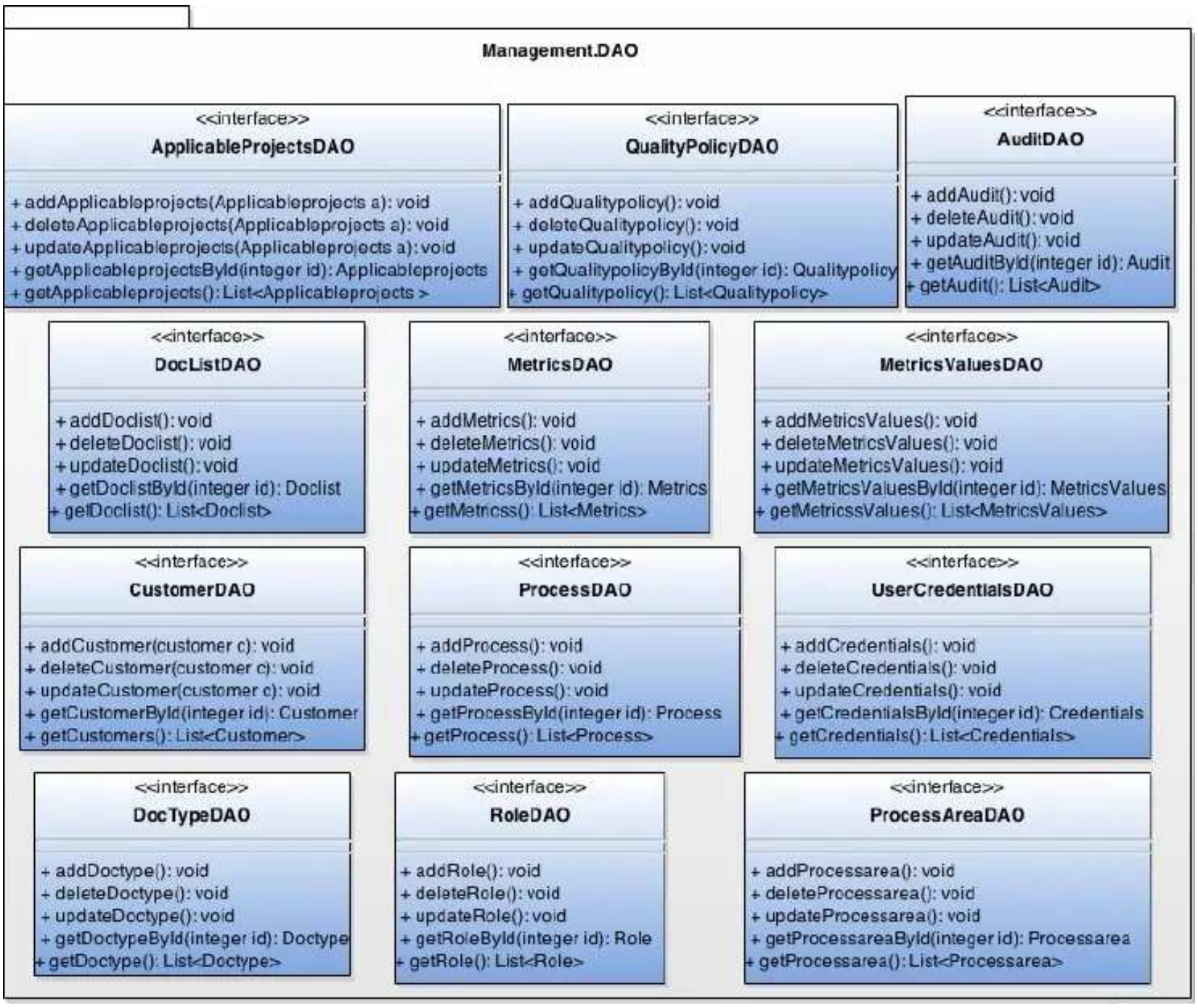


Figure 4.7: Class diagram for the DAO layer

These interface-implementation classes are used by another layer that defines the services provided for the models: the Service layer, explaind next.

4.2.3 Service layer

The Service layer is the responsible of defining the different functionalities of the appliction. It is the core of the buisness logic of the project. It deals with the database through the DAO layer and defianetly uses the model layer.

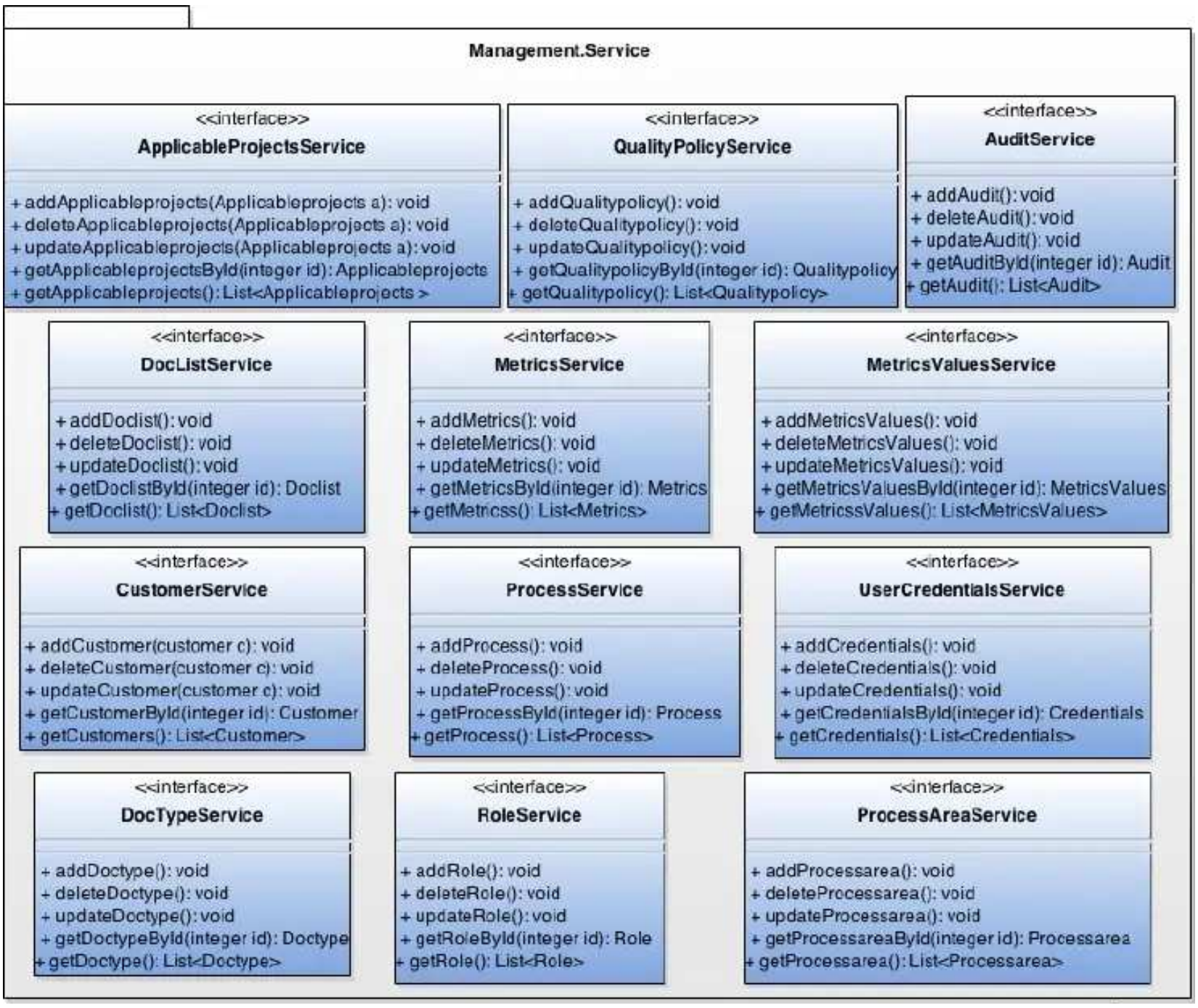


Figure 4.8: Class diagram for the Service layer

the previous figure *(figure 4.8)* demonstrates well the different classes used in this layer. This layer follows the inteface-implementation logic, the same for the DAO layer. This layer, is in interaction with the presentation layer, explained later.

4.2.4 Presentation layer

This layer contains the set of managed beans of our application. A managed bean is a class that works as a "controller" for the components of the user's interfaces. Behind every component displayed for the user, there is a specefic managed bean, that handle its actions.
To seperate more the different functionalities within this layer, we divided it to two packages, the first "data" which containes the managed beans responsible for the beans that handle the data management.

The next figure *(figure 4.10)* presents the set of managed bean in the package "Data"

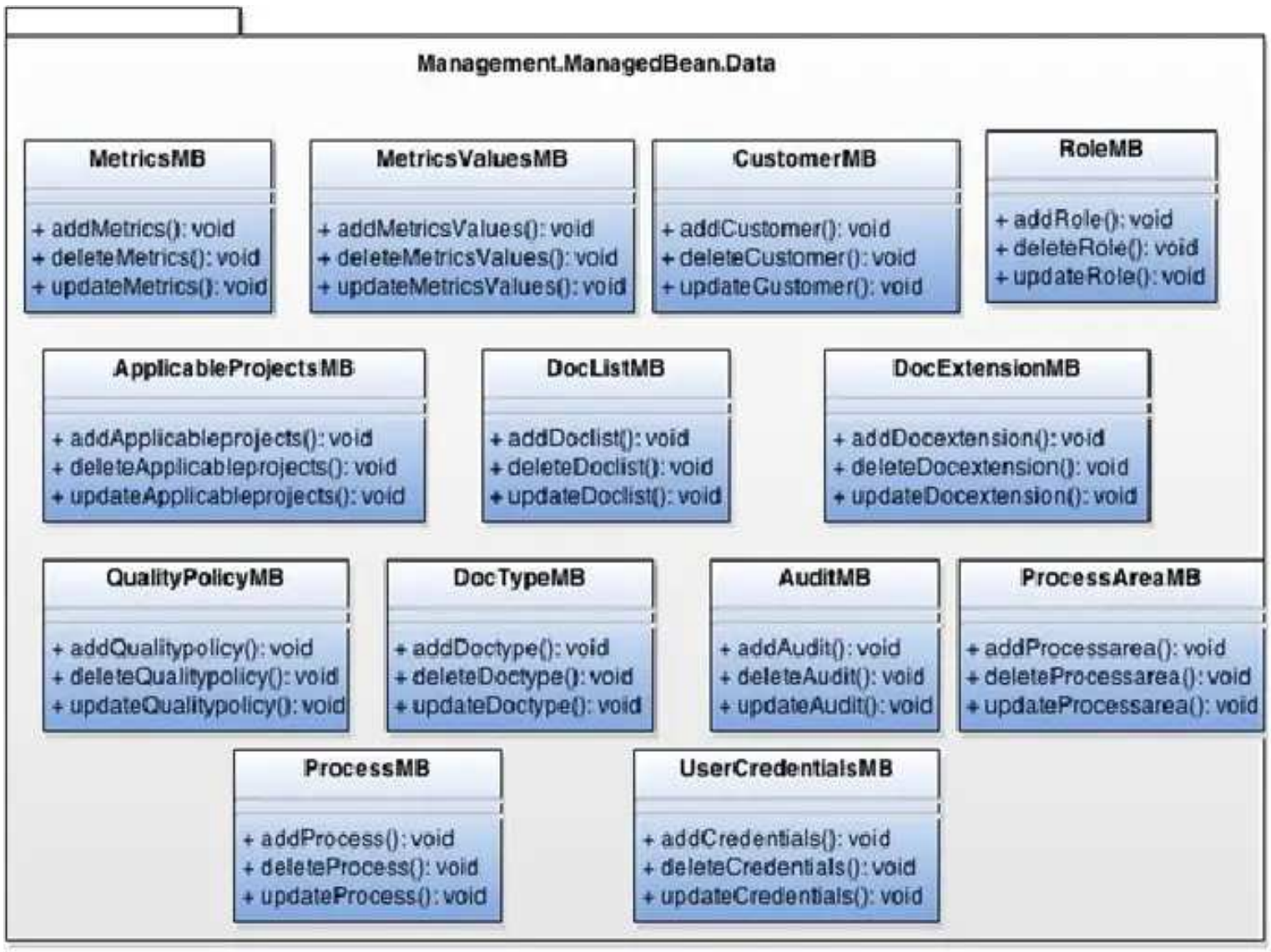


Figure 4.9: Class diagram for the Presentation layer, data package

the second package "Display" contains the managed beans responsible for the display. The next figure *(figure 4.9)* illustrate these special classes within the package "Display".

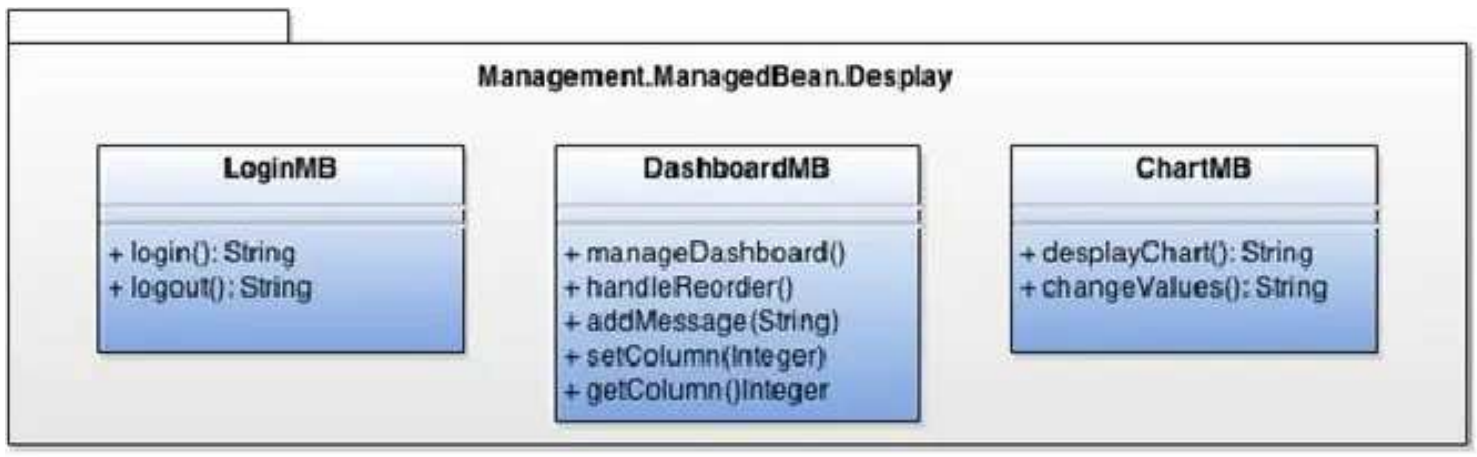


Figure 4.10: Class diagram for the Presentation layer, presentation package

The latter classes use, via the interfaces, the different functionalities of the Service layer, and use the classes of the Model layer. Moreover they manage the interactions with between the user and the application.

Conclusion

All along this chapter we displayed the general architecture of our application, and showed some of the detailed designs for the Buisness Layer and the database. In the next chapter we will be presenting and exposing the technologies that helped during the process of creating our application.

Chapter 5

Achievements

Introduction

In this chapter we will be presenting and arguing the different technologies that were chosen to implement the application according to the design that was presented in the previous chapter. Then we will display some of the interfaces of the application. We finish by presenting the schedule that we followed during the internship.

5.1 Developing environment

In this part we will start by listing the set of software and hardware environment that helped to achieve the desired application.

5.1.1 The hardware environment

During our internship we developed our application using our laptop. It is characterized by:

- 2GO of RAM
- 2 MO of cache memory
- Running Windows7 32 bits
- Installed JVM

5.1.2 The software environment

During the realization of our project we had the chance to work with the next software environment.

5.1.2.1 Main Programs

Here we list the most important software environment that were used.

5.1.2.1.1 Eclipse Indigo. Eclipse[N4] is a free integrated development environment (IDE). It contains a base workspace and an extensible plug-in system for customizing the environment. Written mostly in Java, Eclipse can be used to develop applications. By means of various plugins, Eclipse may also be used to develop applications in other programming languages. The plugins system in Eclipse was useful particularly in our project, because we used the plugin "Maven" to help building and downloading the needed libraries for our project. And our choice was for the Eclipse Indigo because it works and cooperates fine with Maven.

5.1.2.1.2 Maven (plugin). Maven[N5] is a build automation tool used primarily for Java projects. Maven addresses two aspects of building software: First, it describes how software is built, and second, it describes its dependencies. Contrary to preceding tools like Apache Ant it uses conventions for the build procedure, and only exceptions need to be written down. The most advantageous thing about Maven is that it can easily be added to Eclipse, as a plugin, to help handling the dependencies.

5.1.2.1.3 Tomcat (version 7.0). Tomcat [N6] is an open source Java application server and servlet container developed by the Apache Software Foundation (ASF). Tomcat implements the Java Servlet and the JavaServer Pages (JSP) specifications from Sun Microsystems, and provides a "pure Java" HTTP web server environment for Java code to run in. The servlet container manages Servlets (knows where physically Java classes, for which the URL call ...), and executes them when requested. In our project we used Tomcat7 as a servlet container because it's light for using spring framework.

5.1.2.1.4 Database Management System MySQL and MySQL Workbench. Our choice of MySQL [N7] as the relational database management system (RDBMS) for our application was based on the fact that it is very fast, reliable, scalable, and easy to use. MySQL is an Open

Source SQL database management system, is developed, distributed, and supported by Oracle Corporation. It is shipped with no user interface tools to administer MySQL databases or manage data contained within the databases. Users may use the included command line tools, or use MySQL "front-ends", desktop software and web applications that create and manage

MySQL databases, for our project we used MySQL Workbench to manage the data within the database.

5.1.2.1.5 LDAP and LDAP Explorer Tool. LDAP [N8], Lightweight Directory Access Protocol, is an Internet protocol that email and other programs use to look up information from a server. In our case the company uses LDAP as a light database to contain the names, credentials and roles of the users of the application. And that is because it is very easy to access and there is not big amount of updating. For our application it's the source of information about the users. And before we deployed our project in the company's server we simulated the work of an LDAP server with the program "LDAP Explorer Tool"

5.1.2.2 Other Programs

We also used in the process of developing our application other programs such as:

- Hibernate tools, to generate the classes and their annotations from an existing database (reverse engineering).
- Pencil, a tool to draw the story board of the application. It is a helps to better understand the user's need before starting the development phase.
- GanntProjects, used to draw the schedule in a more expressive way.

5.2 Technical choices: Development languages and paradigms

In this section we will discuss the technical choices we madein the way of achieving our application. We will start by presenting the languages used in the development of the project, and then we will shed the light on the standards we used. Afterwords we will defend our choice for the framework we used in the presentation and persistence layers.

5.2.1 Development languages

During our internship we had the chance to use the following languages.

- Java: is a class-based, object-oriented language. Specifically designed to have as few implementation dependencies as possible. It is intended to let application developers "write once, run anywhere" (WORA), meaning that code that runs on one platform does not need to be recompiled to run on another.
- XML (Extensible Markup Language): is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable. We used it in our project to define the filters and the other functionalities of Spring and to present the dependencies.
- HQL (Hibernate Query Language): the syntax is quite similar to database SQL language. The main difference is HQL uses class name instead of table name, and property names instead of column name. We used it in the project to have access to data in the database from the Java code.
- HTML5 (HTML+ CSS+ JavaScript+ jQuery)

5.2.2 Development standards

As we mentioned in the previous chapter, our work was the result of putting together some of the well-known design patterns in the web application world.

- DAO
- MVC
- Spring IoC

These design patterns were fully introduced in the latter chapter.

5.2.3 Development frameworks

After having a deep thought on how to implement the multi-layered architecture we defined in the latter chapter, and to meet the QM responsible expectation of using Spring and Hibernate, we settled on the following frameworks:

- For the model layer, JPA specification, we put the Hibernate framework.
- For the controller layer we put Spring framework.

- For the view layer we choose JSF2 specification, namely the Primefaces framework.

And the putting-together diagram is shown in the next figure *(figure 5.1)*.

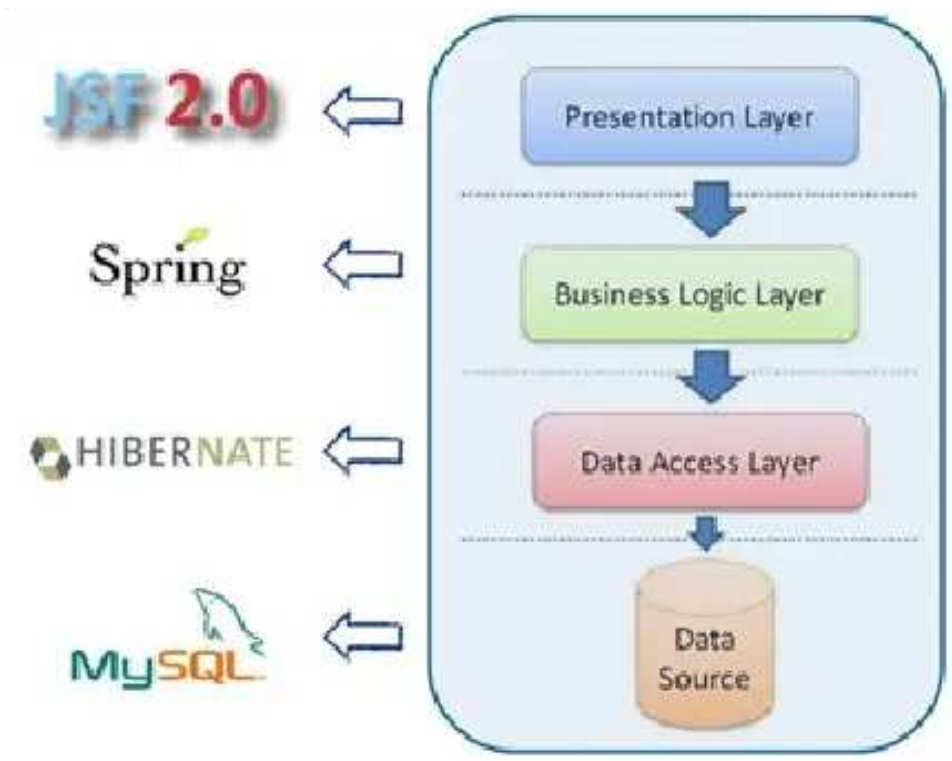


Figure 5.1: Sum up diagram for the choosen frameworks

Besides the fact that, as early as the requirement specification stage, it was imposed on us to use the Spring and Hibernate frameworks. But this choice has strong reasons behind it. Next, we will analyze the benefits of these frameworks and how they are in a perfect fit with our project.

5.2.3.1 Spring Framework

Using J2EE by itself alone (naked) would be not that beneficial as it ought to be, because there is a few problems with these standards [N9] . First of all, the usage of these standards is too complex. And writing a component (EJB: Enterprise Java Bean) requires writing a set of xml files (deployment descriptors), home interfaces, remote/local interfaces, etc. Moreover, there is the 'look-up' problem, when a component requires another component, the components itself is responsible for looking up the components it depends upon. Unfortunately, this look-up happens by name, so the name of the dependency must be hardcoded in the component (code or deployment descriptor). On top of that, letting components communicate with each other from J2EE application servers of different vendors is almost always problematic. So the

components became heavy weight. And that is why developers were using a Plain Old Java Objects (POJO) in coding the business logic. But with this solution one cannot benefit from the services provided by the container of J2EE, such as transaction management, remoting, etc. Here is why we choose the spring framework, to have the benefits of J2EE and POJO.

In fact Spring is a container that is used to wire things using dependency injection. Spring provides additional services like transaction management and seamless integration of various other technologies. Among the functionalities that we used from the Spring Framework in our project, we have

- Spring Security "login-logout", "Access Control List ACL" to define for each profile its granted permessions, "the session's timeout", "remember me" and the Filters.
- Spring internationalization (to change the language for the user's interface)
- Dependency injection (to assure the independence between the layers)
- Annotations, which can help make the development easier, thus faster.

5.2.3.2 Hibernante Framework

While it is true that Spring is a general-purpose framework that plays different roles in many areas of application architecture namely, the persistence. It does not provide its own persistence framework. Instead, it provides an abstraction layer over Java database Connectevity JDBC, and a variety of Oject/Relationel mapping frameworks, namely in our application: Hibernate. This abstraction allows consistent, manageable data-access implementation [N10] . Hibernate ORM (Hibernate in short) is an object-relational mapping library for the Java language, providing a framework for mapping an object-oriented domain model to a traditional relational database. It also works fine with the Spring framework.

5.2.3.3 PrimeFaces Library

PrimeFaces is an open source component library[N11] for JavaServer Faces, developed by "Prime Technology". It provides a collection of mostly visual components (widgets). These can be used by JSF programmers in addition to the small set of basic components that ship with the core JSF platform to compose the UI for a web application. And specifically we used in our application this library because it is elegant and has some charts that are very useful in our dashboard. These charts have JQuery in their backend.

5.3 Work achievements

In order to be successful, an application should adopt a fluent and easy to follow interfaces. Especially in our case: a web application. We try in this section to present the different interfaces provided by our application.

5.3.1 Access to the application

In order to access to the web application, a user must access to the address of the application, and a login page *(figure 5.2)* will be opened, then he must provide his credentials.



Figure 5.2: Login page

The application checks these credentials in the LDAP and in case they are valid, the application will be opened and display the home page *(figure 5.3)*. And it loads the roles of this user, in order to customize its granted permissions. This permessions will controle what are the funcyionalities that the user will be able to do in the home page.

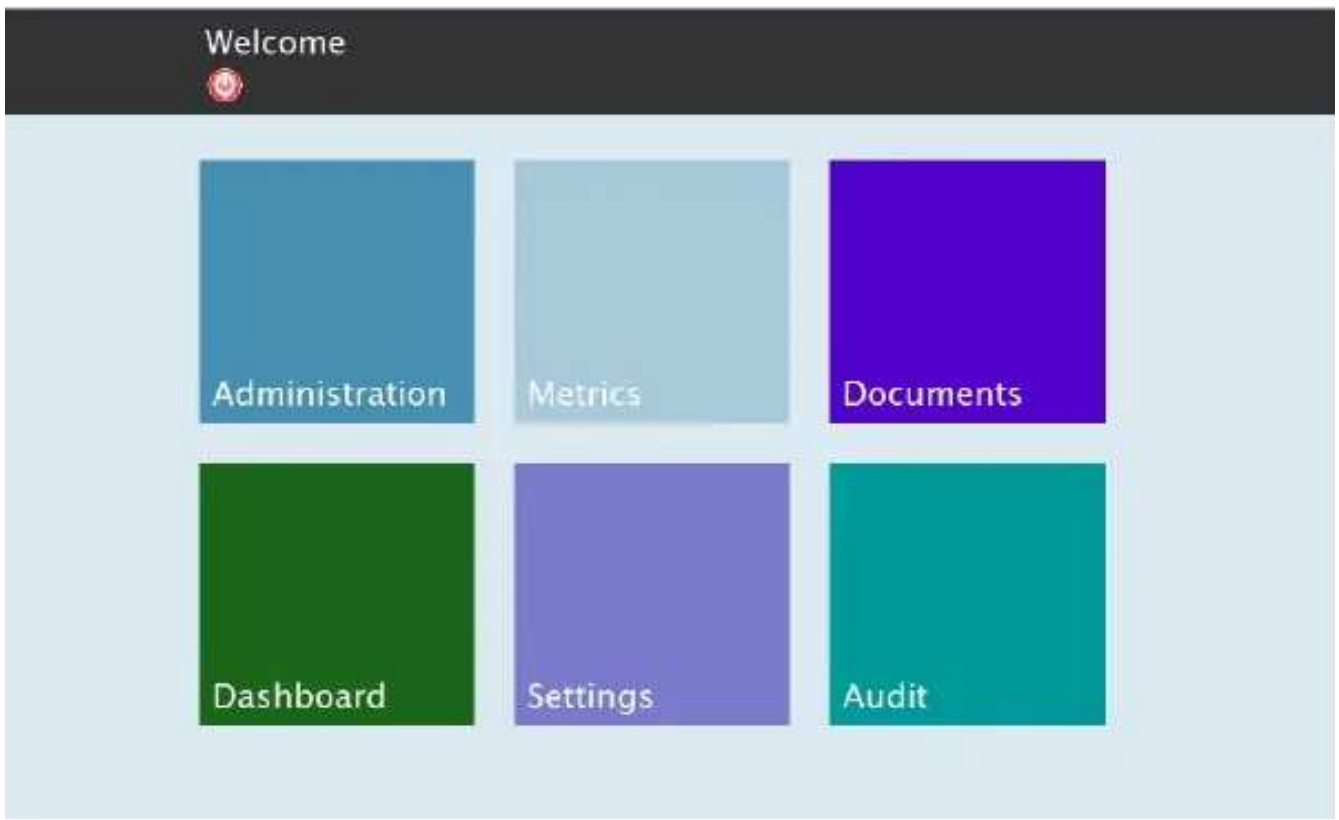


Figure 5.3: Home page

The welcome page contains the different functionalities of the application, with the condition that the user has the right to do it. For instance it permits:

- The logout
- To manage the application if he is an admin
- To check the values of a KPI and update their values
- To track the documents
- To plan an audit

We go into each one of these functionalities in the next paragraphs.

5.3.2 Administration of the application

This interface *(figure 5.4)* is not static, actually it can only be displayed for the admin profile, and it contains a reminder of the user's credentials and a set of information that the admin can manage. For example theire is the information about the Processes and the Process Areas and the Projects...

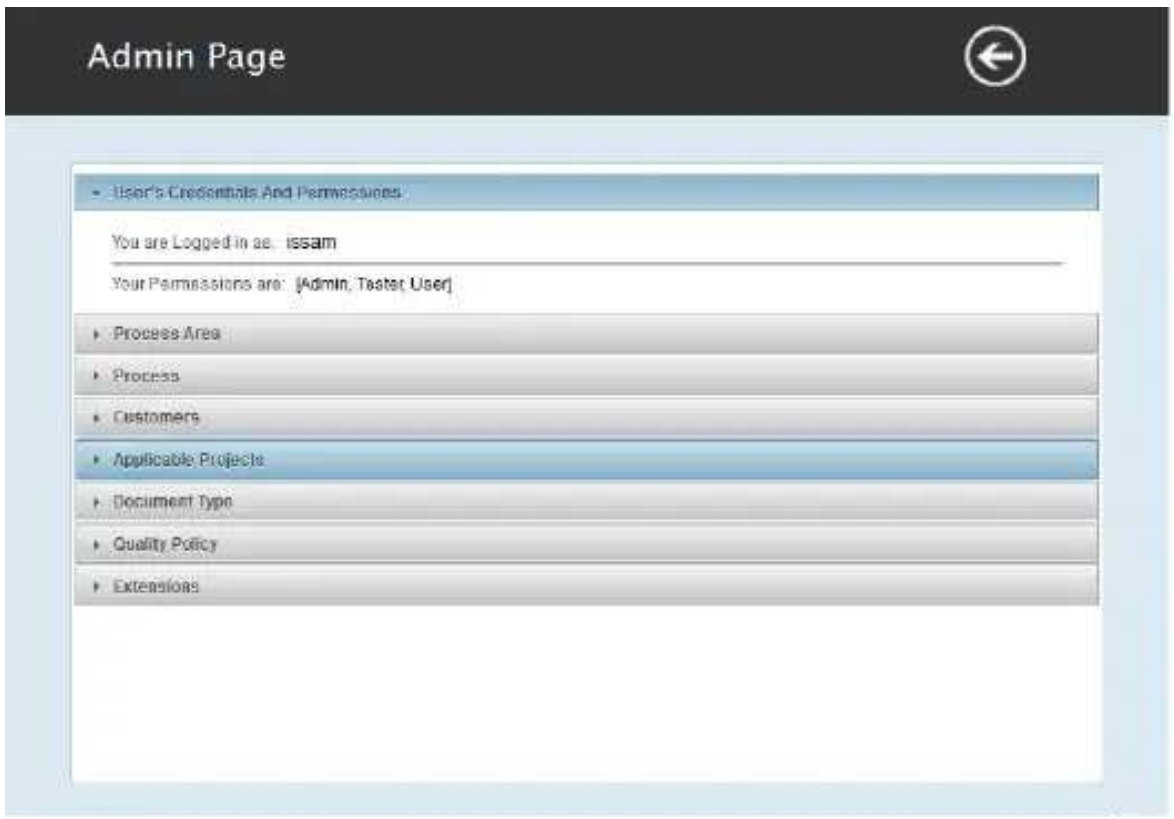


Figure 5.4: Admin page

In fact the user, by using the latter interface, can add a new item to the database, for instance in the figure *(figure 5.5)* he add a new entry for the "Process Area".

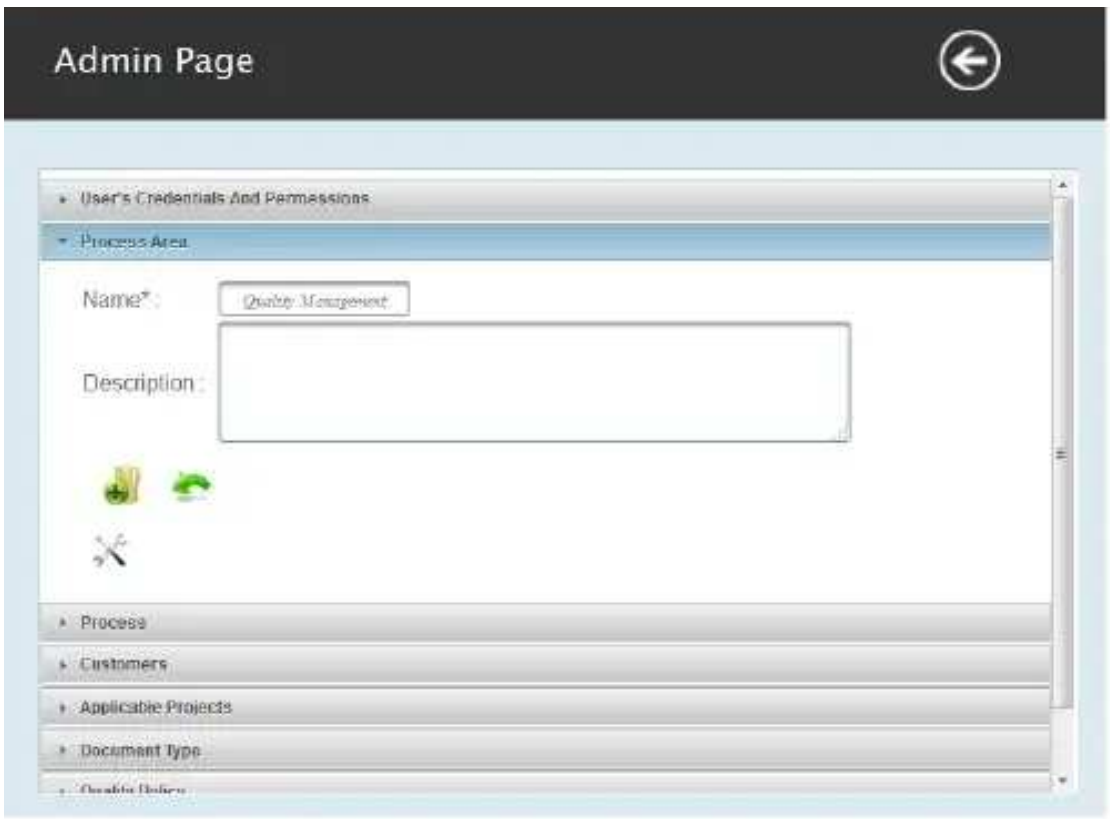


Figure 5.5: Interface to add new information

By clicking on the blue icon, the user can display all the existing elements in the database. For example the figure *(figure 5.6)* that shows all the element for the "Process Area".



Figure 5.6: Interface to display the existing information

And by accessing the previous interface the user can choose between deleting an exisiting element, illustrated by the figure *(figure 5.7)*

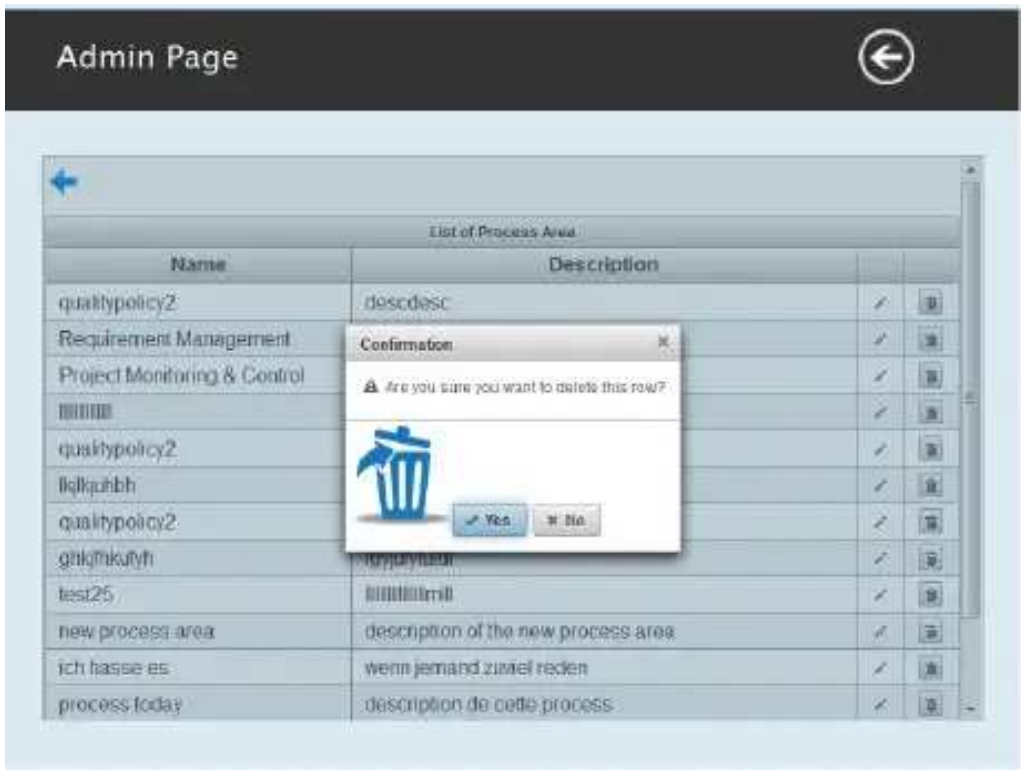


Figure 5.7: Interface to delete information

Or he can update an existing element as the figure *(figure 5.8)* demonstrate.

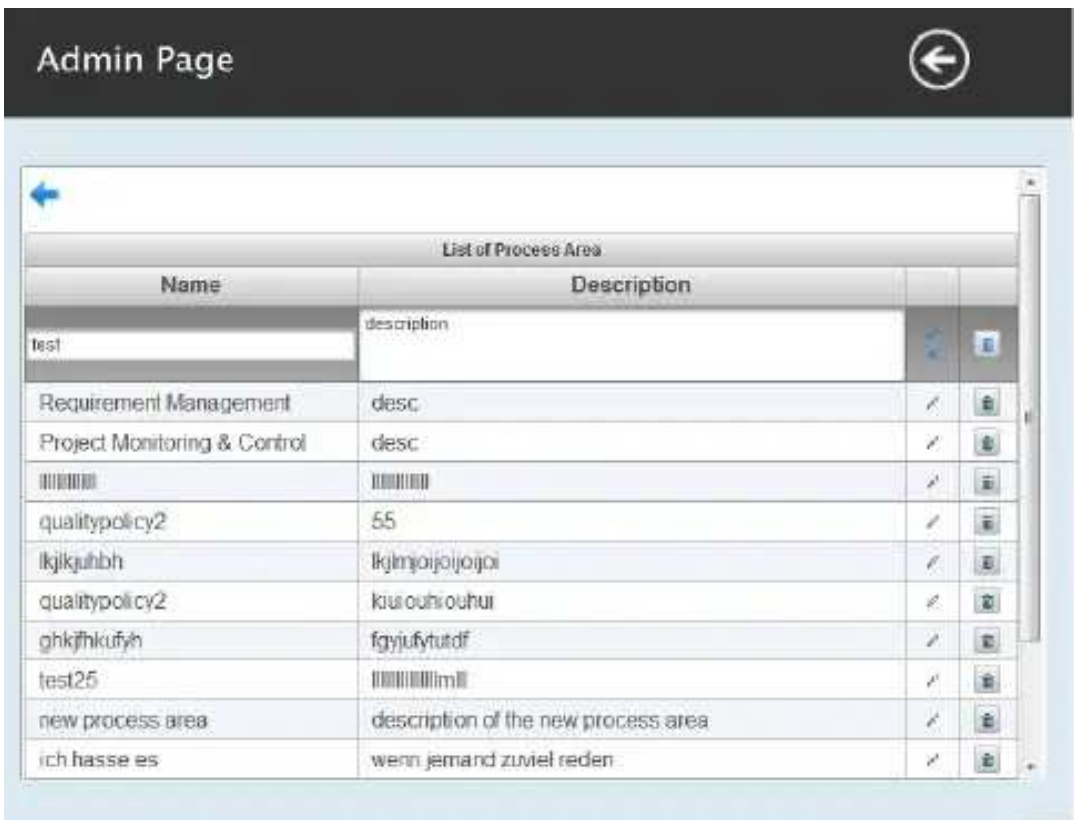


Figure 5.8: Interface to update information

By clicking on the arrow on the right side of the window, the user can get back to the home page and choose another functionality.

5.3.3 Managing the KPIs

From the home page, the user can choose another functionality: the KPI management. In order to set the configuration of the dashboard, the user got to acces the interface of settings *(figure 5.9)* which enables him to choose the settings of his dashboard, by choosing which process or KPIs to be displayed in the next time he opens the dashboard.

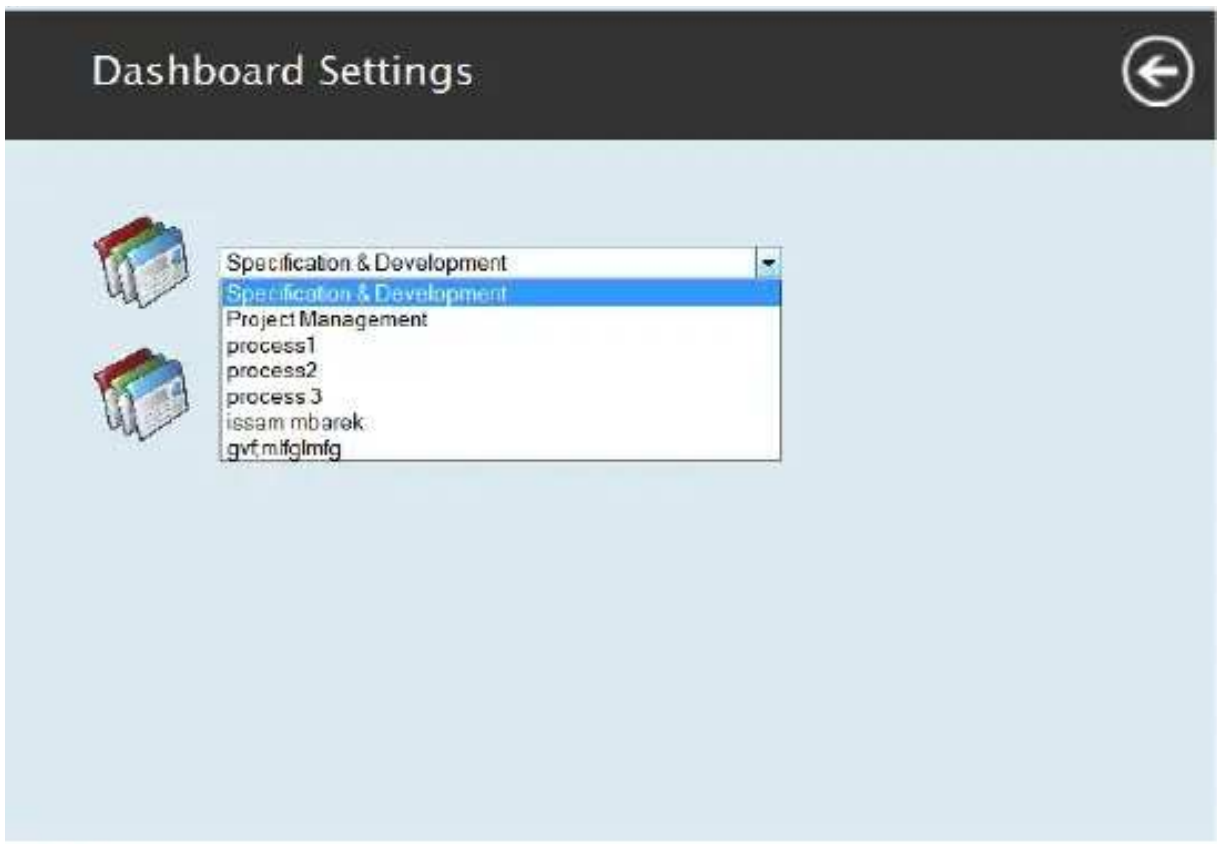


Figure 5.9: Dashboard settings

After setting his preferances, the user can select a specefic KPI to display display the correspondent chart *(figure 5.10)*. This chart containes the values of a specefic year with providing the desired target for each month.

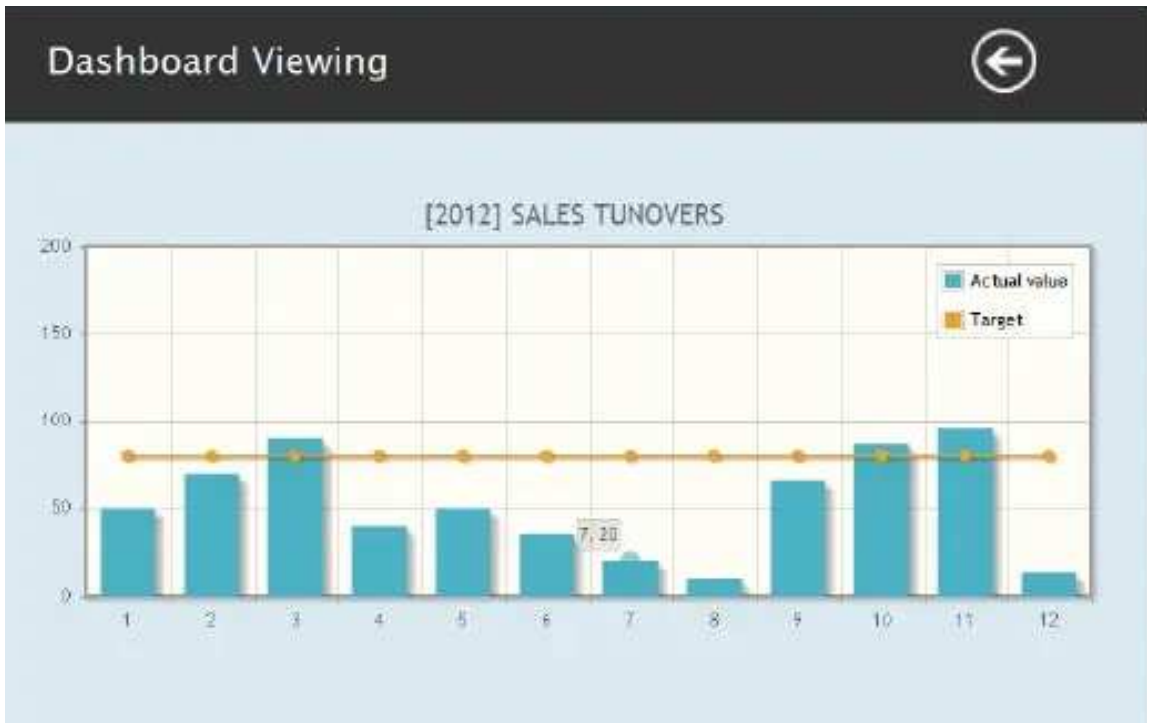


Figure 5.10: Display of the Sales turnover's value for the year 2012

Moreover the application offers the ability to update the values of the KPIs through the next interface *(figure 5.11)*.

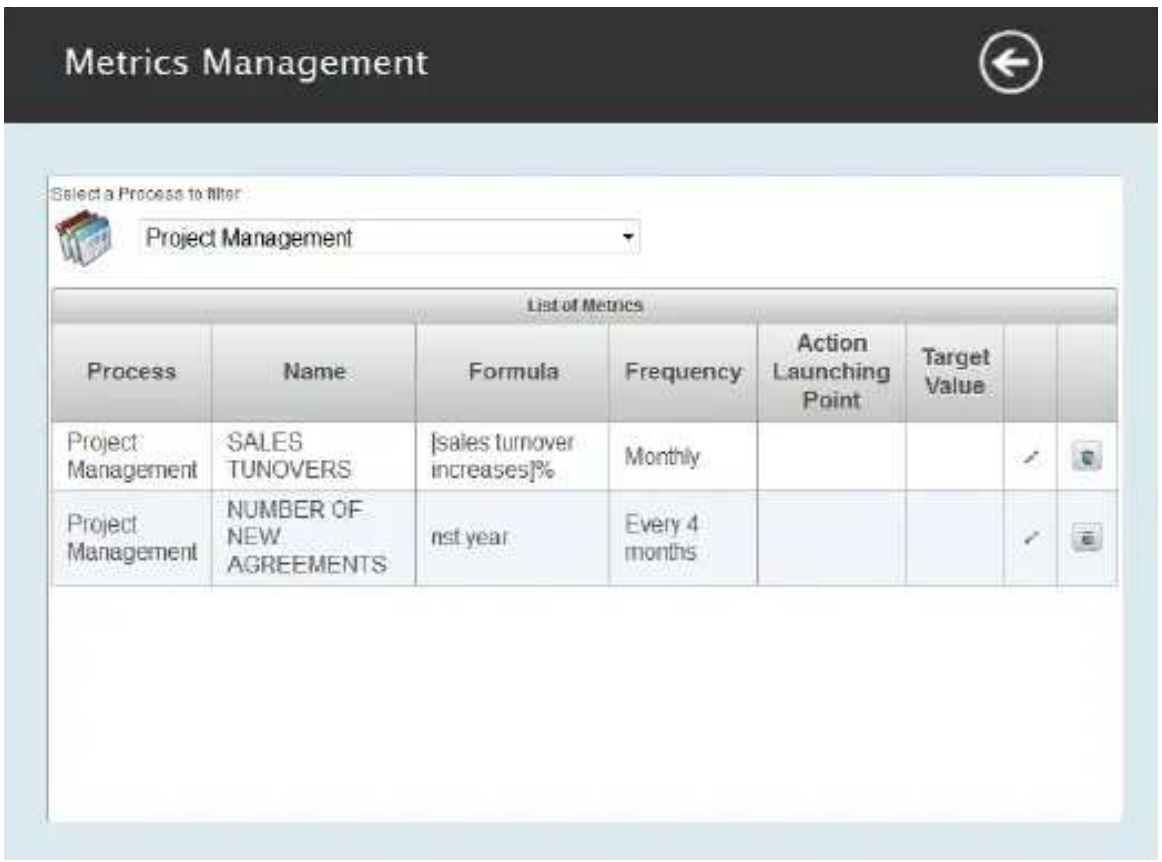


Figure 5.11: Interface to update the KPIs values

5.3.4 Document traceability

Another possibility for the user is to choose the document traceability from the main menu. This interface *(figure 5.12)* enables the user to input the information: name, type version and description of document.

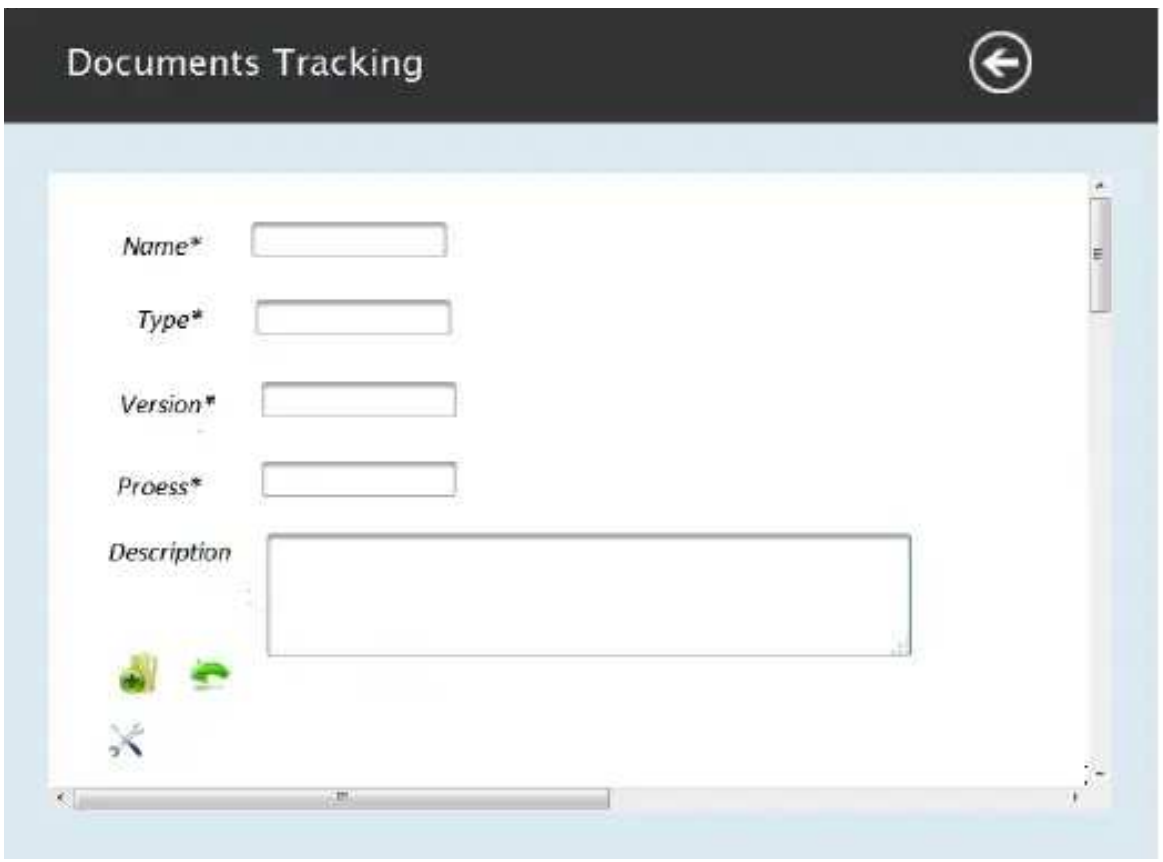


Figure 5.12: Interface to provide the information for document tracking

The application checks that these information are valid and then stock them in the database.

5.3.5 Audit planning

This interface, *(figure 5.13)* , is important for the user to make the planning of the audit. This interface is only displayed for the workers that are entitled to start an audit planning for instance the Responsible of the QM and the executive of the company. The user must provide the information related to the audit namely the audited workers, the auditors and the date, and the application takes care of the announcement to the involved workers.

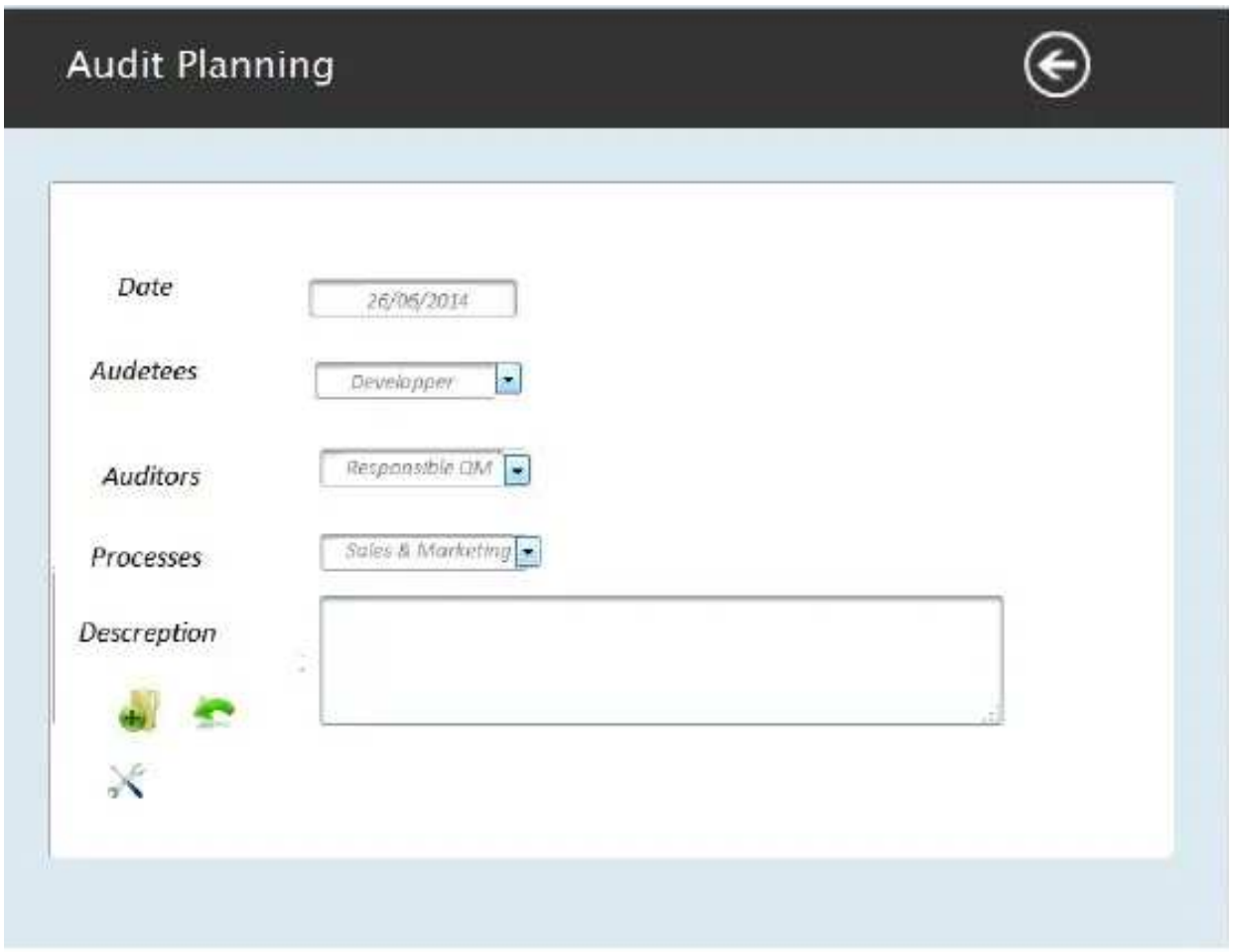


Figure 5.13: Interface to audit planning

5.4 Project's planning

Throughout our internship we tried as much as possible to stick to the schedule. Eventually we had to do some changes. Some phases were longer while others were shorter. The Planning of our internship is as shown in the next figure *(figure 5.14)* .

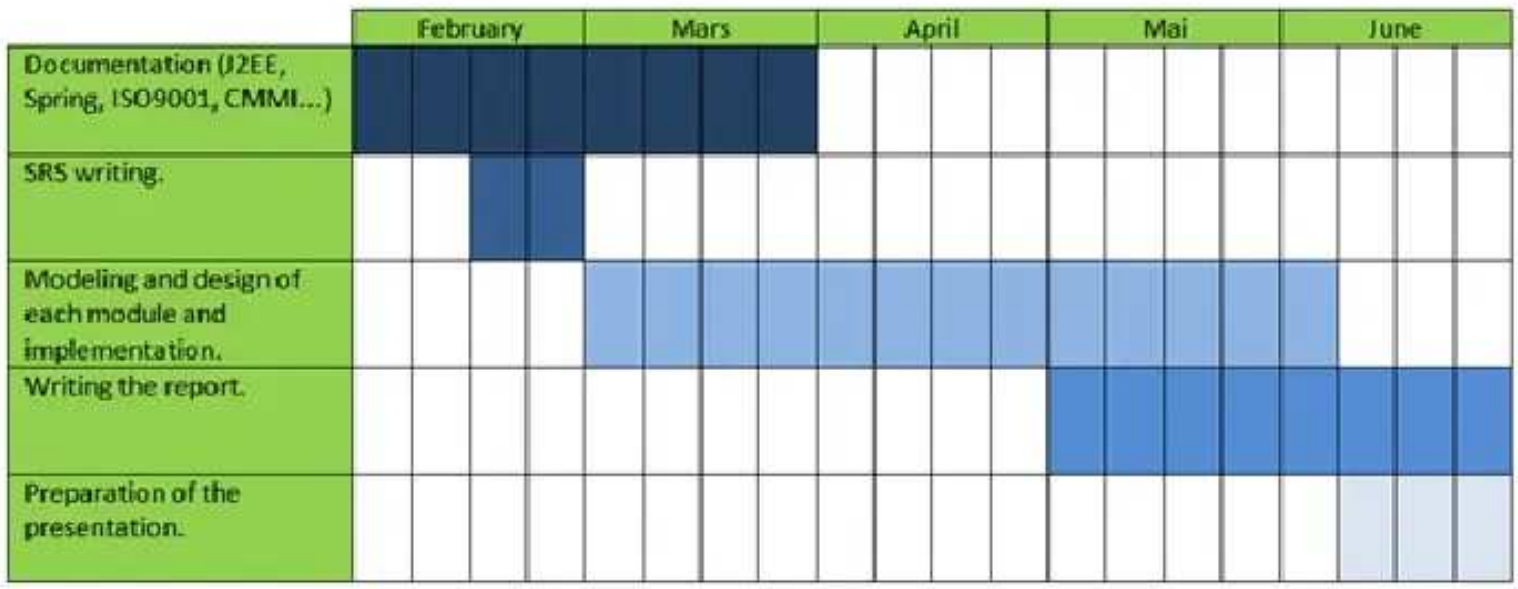


Figure 5.14: Work timeline

We started by a period of documentation, in which we had to learn more about CMMI, ISO 9001, J2EE, Spring, Hibernate.... And then we wrote the Software Requirement Specification (SRS). In order to break down its difficulty, we subdivided the projects into different modules And then we begin the phase of having the requirements collected and then presented by use cases. We began afterwards the design phase and then the programming. We repeated each of the last phases in a cycle for each module apart.

Conclusion

During this chapter we presented the technologies that were chosen to implement our projects. We gave arguments for that choice. We also presented the different frameworks that were used. We finished by displaying the main features and interfaces of our application.

General conclusion and perspectives

In this project, dedicated to obtain the national degree of engineering from the National School for Computer Science (Ecole Nationale des Sciences de l'Informatique ENSI), we designed and developed an application that helps optimizing the Quality Management for the company. This optimization concerned specifically the "tracking of KPIs", the "Audit Planning" and the "Documents Traceability".

The literature review and the study of the state of the art, enabled us to propose our solution to the challenges that Focus, the hosting company is facing. We embraced a semi formal presentation of the QM responsible's requirements. Therefore we used a set of use cases diagrams using the UML.

During the different phases of the development of our application we worked with various technologies, programming languages and frameworks, for instance Java as a backend programming language, Hibernate as a persistence framework, JSF2 and PrimeFaces for the frontend interfaces, and more importantly the Spring Framework as a wrap-up framework for all the other frameworks. And we got benefit from the spring security functionalities. In fact this choice of frameworks had a big influence on the design of our application, thus we made it a web application with multi tiers. And to better implement the application we followed the J2EE and the MVC design patterns, and the Spring layering architecture. Besides we didn't forget throughout this project to follow the Rational Unified Process (RUP) software development process to get closer to the client's vision.

To put it in a nutshell, as it was mentioned, our application aimed for making the Quality Management more efficient, namely the "Document Management" and the "Audit Management" and "Supervising the KPIs with a dashboard".

Nevertheless Quality Management is a wider field than just those functionalities.
So to enhance our application we would propose to add modules that reflect other aspects of the Quality Management System for example the “Management of the Non-conformities”, the “Risk Management” and “Complaint Handling”. This will be possible due to the extendible nature of Spring and the MVC.

Bibliography

[1] C. Escanciano, E. Fernandez, C. Vazquez, *Linkng the firm's technological status and ISO 9000 certification: results of empirical research,* Journal of technovation 22, 2002, p.1.

[2] A. Lester, Project Management, *Project Management, Planning and Control: Managing Engineering, Construction and Manufacturing Projects to PMI,* APM and BSI Standards, 6th ed. (Elsevier Science and Technology Books, November 2006), pp.87-91.

[3] S. Kim, Y.-S. Chung,, *Critical success factors for IS outsourcing implementation from an interorganizational relationship perspective,* Journal of Computer Information Systems 43, 2003, p. 81.

[4] R. Gonzalez, J. Gasco, J. Llopis, *Information systems outsourcing: an empirical study of success factors,* Human Systems Management 29, 2010, pp. 139–151.

[5] C. Schwarz, *Toward an understanding of the nature and conceptualization of outsourcing success,* Journal of Information and Management 51, 2014, p.185.

[6] P.M. Bentler, D.G. Bonett, *Significance tests and goodness of fit in the analysis of covariance structures,* Psychological Bulletin 88, 1980, p. 588.

[7] D.i LIU, "Design and Implementation of High-quality Course Scoring System Based on Struts and Spring and Hibernate Architecture, in "*International Conference of Information Technology, Computer Engineering and Management Sciences* , 2010, p.46.

[8] T. bak, B. sakowicz, A. napieralski, "Development of advanced J2EE solutions based on lightweigjt containers on the example of "E-DEPARTMENT" Application" in *International Conference MIXDES,* Gdynia, POLAND, 2006, p.779.

[9] W. HaiTao, J. BaoXian, "Research Based on Web Development of Spring Integration Framework", in *International Forum on Information Technology and Applications,* 2002, p.2.

[10] W. Jing; C. Yue-feng,XU Feng, "Design and Application of Java Web Software Architecture Based on the SH Middleware", in *The Design of the System for Zhanjiang Meteorological Forecast*, 2010, pp.1,3.

[11] P.Borges, P. Monteiro and R. J. Machado, "Tailoring RUP to Small Software Development Teams", in *37th EUROMICRO Conference on Software Engineering and Advanced Applications,* 2011, pp.1,2.

[12] A.J. Walker, "Quality Management applied to the Development of a National Checklist for IS0 9001 Audits for Software", 1997, pp.6,7,8.

[12] P. Monteiro, P. Borges, R. J. Machado and P. Ribeiro "A Reduced Set of RUP Roles to Small Software Development Teams",in ICSSP, Zurich, Switzerland 2012, pp.190,191.

[13] S. Few , *Information Dashboard Design: the Effective Visual Communication of Data* o'reilly editions, pp.2,3,4.

[14] M. Habib, S. Ahmed, A. Rehmat, M. Jahan Khan, S. Shamail ,*Blending Six Sigma and CMMI -An Approach to Accelerate Process Improvement in SMEs* IEEE publications (2008), p.1.

[15] H. Joon Moon, S. Hyun Lee, S. Jun Yoo, E. Jung Yu, C. Seong Leem, " A KPI-based Performance Assessement Framework for Kurean e-Gouvernment" in *Second International Conference on Future Generation Communication and Networking Symposia* (2008), p.2.

[16] G. Xu, H. Hu, P. Yu, J. Lv, P. Qu, M. Zhu, " Supporting Flexibility of the CMMI Process Framework with a Multi-layered Process Model" in *10th Web Information System and Application Conference* (2013),pp.1,2,4,5,6.

[17] J. H. Brown, J. Watts, " Enterprise engineering: building 21st century organizations" in *Viewpoint journal* Butterworth-Heinemann Ltd, Vol 1 No 5 (1992), pp 1,2.

Nethography

[$N1$] Company over view. In focus-corporation.com [online].
Available: http://www.focus-corporation.com/company/company-profile
[2014, june 3].

[$N2$] Company history. In focus-corporation.com [online].
Available: http://www.focus-corporation.com/company/company-history
[2014, june 3].

[$N3$] Quality. In focus-corporation.com [online].
Available: http://www.focus-corporation.com/company/quality
[2014, june 3].

[$N4$] Quality. Eclipse Indigo. In eclipse.org [online].
Available: http://www.eclipse.org/indigo/
[2014, june 3].

[$N5$] Available plugins. In maven.apache.org [online].
Available: http://maven.apache.org/plugins/
[2014, june 3].

[$N6$] Apache tomcat 7. In tomcat.apache.org [online].
Available: http://tomcat.apache.org/tomcat-7.0-doc/windows-service-howto.html
[2014, june 3].

[$N7$] What is MySQL. In dev.mysql.com [online].
Available: http://dev.mysql.com/doc/refman/4.1/en/what-is-mysql.html
[2014, june 3].

[$N8$] What is LDAP. In gracion.com [online].
Available: http://www.gracion.com/server/whatldap.html
[2014, june 3].

[$N9$] Introduction to spring framework. In docs.spring.io [online].
Available: http://docs.spring.io/spring/docs/3.1.0.M1/overview.html
[2014, june 3].

[$N10$] ORM overview. Tutorialspoint.com [online].
Available: http://www.tutorialspoint.com/hibernate/ormoverview.htm
[2014, june 3].

[$N11$] Getting started. In primefaces.org [online].
Available: http://www.primefaces.org/gettingStarted
[2014, june 3].

[$N12$] ISO 9000 - Quality management. In iso.org [online].
Available: www.iso.org/iso/home/standards/management-standards/iso9000.htm
[2014, june 26].

[$N12$] cmmi frequent question. In cmmifaq.info [online].
Available: http://www.cmmifaq.info/
[2014, june 26].

Appendix A

ISO 9000- Quality management

The ISO 9000 family addresses various aspects of quality management and contains some of ISO's best known standards. The standards provide guidance and tools for companies and organizations who want to ensure that their products and services consistently meet customer's requirements, and that quality is consistently improved.

Standards in the ISO 9000 family include:

- ISO 9001:2008 - sets out the requirements of a quality management system
- ISO 9000:2005 - covers the basic concepts and language
- ISO 9004:2009 - focuses on how to make a quality management system more efficient and effective
- ISO 19011:2011 - sets out guidance on internal and external audits of quality management systems.

A.1 ISO 9001:2008

ISO 9001:2008 sets out the criteria for a quality management system and is the only standard in the family that can be certified to (although this is not a requirement). It can be used by any organization, large or small, regardless of its field of activity. In fact ISO 9001:2008 is implemented by over one million companies and organizations in over 170 countries [N12].

This standard is based on a number of quality management principles including a strong customer focus, the motivation and implication of top management, the process approach and continual improvement. Using ISO 9001:2008 helps ensure that customers get consistent, good

quality products and services, which in turn brings many business benefits. Certification to ISO 9001:2008

Checking that the system works is a vital part of ISO 9001:2008. An organization must perform internal audits to check how its quality management system is working. An organization may decide to invite an independent certification body to verify that it is in conformity to the standard, but there is no requirement for this. Alternatively, it might invite its clients to audit the quality system for themselves.

A.2 Support for implementing ISO 9001:2008

ISO 9001:2008 is published by ISO technical committee (TC) ISO/TC 176, sub-committee 2. When the standard was revised and updated in 2008 the TC prepared some guidance documents to help organizations and companies implement the revised version of the standard.

Appendix B

Capability Maturiy Modal Integration: CMMi

CMMI stands for "Capability Maturity Model Integration". It's the integration of several other CMMs (Capability Maturity Models). The CMMI is a framework for business process improvement. In other words, it is a model for building process improvement systems. In the same way that models are used to guide thinking and analysis on how to build other things [N13] (algorithms, buildings, molecules), CMMI is used to build process improvement systems. It is not an engineering development standard or a development life cycle.

There are currently three "flavors" of CMMI called constellations. The most famous one is the CMMI for Development, "DEV". It has been around in one version or another for roughly 10 years and has been the subject of much energy for over 20 years when including its CMM ancestors.

More recently, two other constellations have been created: CMMI for Acquisition, "ACQ", and CMMI for Services, "SVC". All constellations share many things, but fundamentally, they are all nothing more than frameworks for assembling process improvement systems. Each constellation has content that targets improvements in particular areas, tuned to organizations whose primary work effort either:

- Develops products and complex services, and/or
- Acquires goods and services from others, and/or
- Provides/ delivers services.

NONE of the constellations actually contain processes themselves. None of them alone can be used to actually develop products, acquire goods or fulfill services. The assumption with all CMMIs is that the organization has its own standards, processes and procedures by which they actually get things done [N13]. The content of CMMIs are to improve upon the performance of those standards, processes and procedures and not to define them.

Having said that, it should be noted that there will (hopefully) be overlaps between what any given organization already does and content of CMMIs. This overlap should not be misinterpreted as a sign that CMMI content *is*, in fact, process content. It can't be over-emphasized, CMMIs, while chock-full-o examples and explanations, do not contain "how to" anything other than building improvement systems. The overlap is easy to explain: activities that help improve a process can also be activities to effectively perform a process, and, not every organization performs even the basic activities necessary to perform the process area well. So, to one organization, what seems trivial and commonplace, to another is salvation from despair.

Another way to look at CMMIs are that they focus on the business processes of developing engineered solutions (DEV), acquiring goods and services (ACQ) and delivering services (SVC). To date, CMMI has most widely applied in software and systems engineering organizations. Now, with the expansion of the constellations, where it is applied is a significantly distinct matter from being anything even remotely akin to a standard or certification mechanism for the engineering, methods, technologies, or accreditation necessary to build stuff, buy stuff or do stuff. If an organization chose to do so, CMMI could be applied in the construction or even media production industries.

Putting it all together: CMMI is a model for building process improvement systems from which organizations will abstract and create process improvement solutions that fit their unique environment to help them improve their operational performance.